

Dynamic Malware Analysis of Course Sample 2

financials-xls.exe

Technical report: packing validation, controlled runtime observation and network correlation

Student ID	[C11/26/EHIT/17330]
Course / laboratory	CIP-A106, Laboratory 02: Dynamic Malware Analysis
Sample	Course-provided Malware Sample 2: financials-xls.exe
Analysis date	1 October 2026 (VM clock)
Report date	1 October 2026
Analysis platform	Oracle VirtualBox VM "windows11" (FLARE-VM on Windows 11), Host-only network with FakeNet-NG simulated services
Handling	All handling and execution took place inside the isolated VM. The sample was not submitted to any public service. The VM was restored to the clean baseline snapshot at the end of the exercise.
Deliverable	CIP-A106_Lab02_StudentID.zip: this technical report (with embedded evidence appendix), the evidence log (Appendix A) and the exported evidence screenshots

Contents

- 1 Executive summary
- 2 Environment preparation (Section A)
- 3 Packing validation and approved unpacking (Section B)
- 4 Controlled runtime observation (Section C)
- 5 Network and behaviour correlation (Section D)
- 6 Evidence-based findings (Section E)
- 7 Post-exercise integrity, preservation and restoration
- Appendix A Evidence log
- Appendix B Figure index
- Appendix C Deviations, errors and corrections

1 Executive summary

financials-xls.exe is a 43,520-byte, 32-bit Windows GUI executable that is packed with UPX. Its name and Excel icon suggest a spreadsheet, but when it ran inside the isolated VM it behaved as follows:

- **Packing (static):** three tools and the packer itself agree that the file is UPX-packed, and an approved unpack with UPX 5.2.0 produced a 57,344-byte file with different section layout, lower entropy and a much larger import list.
- **Execution:** the sample was run three times in one monitoring window (20:49 to 20:57 VM time). In the documented third run (PID 2868) it started at 20:57:10, showed a fake "Your computer is infected!" spyware alert and exited by itself at 20:57:12.
- **Network:** on each run it sent one HTTP GET to the hard-coded address 69.50.175.181 on port 80 (/download.php with advid=00000717 and varying u and p values; Host header download.bravesentry.com). FakeNet-NG intercepted all traffic, so no real server was contacted.
- **System changes:** two new HKCU Run values appeared during the window: "con" pointing at the sample itself, and "Windows update loader" pointing at C:\Windows\xpupdate.exe.
- **Not established:** file-system activity, the full process tree, anti-analysis behaviour and the existence of xpupdate.exe could not be verified because the Process Monitor capture was not preserved and Regshot file scanning was off.

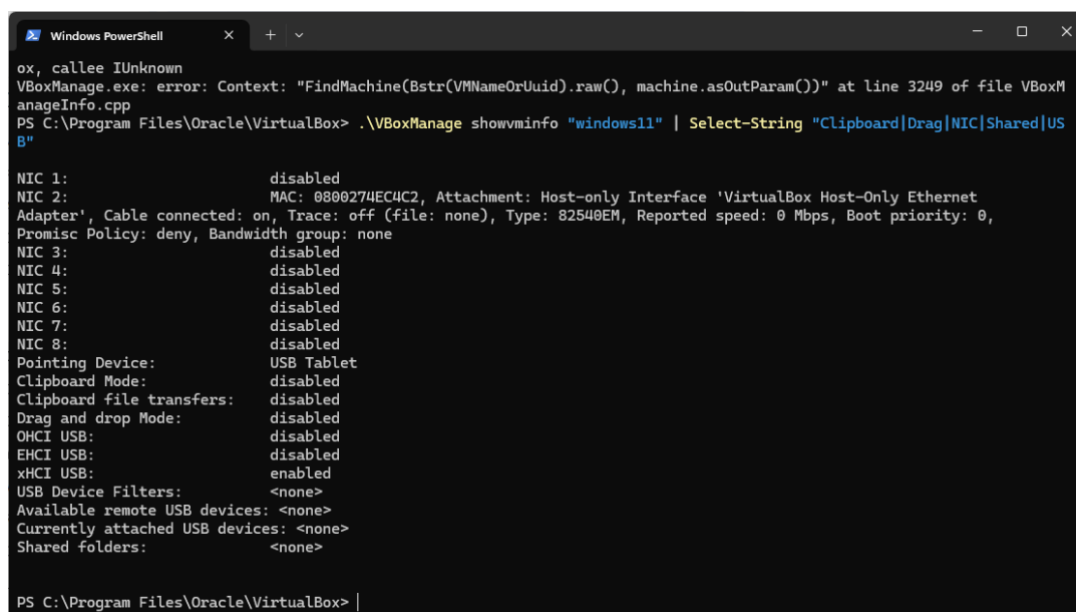
Area	Result	Confidence
Packing	UPX packed (DIE UPX 3.92 signature; UPX0/UPX1 sections; UPX1 entropy 7.844; upx -t [OK]); unpacked to 57,344 bytes	High
Process	Run 3: PID 2868, lifetime about 2 s, fake spyware alert shown; runs 1 and 2 (PIDs 1976 and 1600) seen only in the FakeNet log	High (run 3); Medium (runs 1 and 2)
Network	Three HTTP GETs to 69.50.175.181:80, Host download.bravesentry.com, intercepted by FakeNet-NG	High that requests occurred; Low on purpose
Persistence	HKCU Run: con = C:\L\o\financials-xls.exe; Windows update loader = C:\Windows\xpupdate.exe	Medium-High (con); Medium (xpupdate)
Likely capability	Scareware or rogue-security-style downloader that contacts a remote server and sets up Run-key persistence	Medium overall; Low on any downloaded payload

2 Environment preparation (Section A)

This section records the isolation controls, the environment, the evidence tools, the baselines taken before any execution and the hash of the original sample. Times in PowerShell captures are the FLARE prompt timestamps, which show when each prompt was drawn, so the command was entered at or shortly after that time. Times labelled "host" come from the VirtualBox interface on the host machine.

2.1 Isolation controls

Control	Observed state	Evidence
Hypervisor / VM	Oracle VirtualBox; VM "windows11" (FLARE-VM), Windows 11 64-bit	Figure 2
Network	Host-only only: NIC 2 attached to the VirtualBox Host-Only interface (Intel PRO/1000 MT Desktop, cable connected, promiscuous policy deny); NIC 1 and NICs 3 to 8 disabled	Figure 1, Figure 2, Figure 26
Internet access	None. Simulated services were provided inside the guest by FakeNet-NG, and outbound connections were redirected by its Diverter	Figure 38
Shared clipboard	Disabled (including clipboard file transfers)	Figure 1, Figure 3
Drag and drop	Disabled	Figure 1, Figure 3
Shared folders	None	Figure 1, Figure 26
USB	xHCI controller enabled; USB device filters none; remote and attached USB devices none (OHCI and EHCI disabled). The controller itself was not disabled, so USB isolation rests on no device being attached.	Figure 1, Figure 2
Snapshot	Lab02_PreExecution_Baseline taken before the working folders were created and before the sample was extracted (see 2.3)	Figure 4
Authorisation	[Instructor authorisation reference to be added by the student]	n/a



```
Windows PowerShell
ox, callee IUnknown
VBoxManage.exe: error: Context: "FindMachine(Bstr(VMNameOrUuid).raw(), machine.asOutParam())" at line 3249 of file VBoxM
anagerInfo.cpp
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage showvminfo "windows11" | Select-String "Clipboard|Drag|NIC|Shared|US
B"

NIC 1: disabled
NIC 2: MAC: 0800274EC4C2, Attachment: Host-only Interface 'VirtualBox Host-Only Ethernet
Adapter', Cable connected: on, Trace: off (file: none), Type: 82540EM, Reported speed: 0 Mbps, Boot priority: 0,
Promisc Policy: deny, Bandwidth group: none
NIC 3: disabled
NIC 4: disabled
NIC 5: disabled
NIC 6: disabled
NIC 7: disabled
NIC 8: disabled
Pointing Device: USB Tablet
Clipboard Mode: disabled
Clipboard file transfers: disabled
Drag and drop Mode: disabled
OHCI USB: disabled
EHCI USB: disabled
xHCI USB: enabled
USB Device Filters: <none>
Available remote USB devices: <none>
Currently attached USB devices: <none>
Shared folders: <none>

PS C:\Program Files\Oracle\VirtualBox> |
```

Figure 1. VBoxManage showvminfo output for VM windows11: NIC 1 and NICs 3-8 disabled; NIC 2 attached to the Host-only interface; clipboard, clipboard file transfers and drag-and-drop disabled; shared folders none; xHCI USB enabled with no filters and no attached devices. The first line shows an earlier attempt that failed with a FindMachine error before the command was re-run.

[Source: baseline_configuration_of_the_windows_11_from_powershell.png]

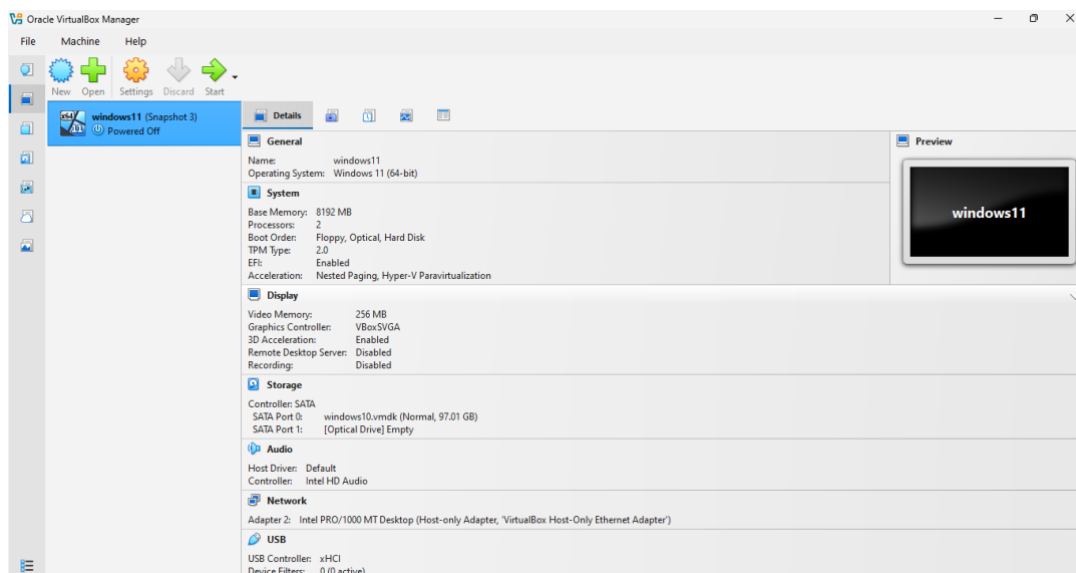


Figure 2. VirtualBox Manager details for windows11 (Windows 11 64-bit, 8192 MB RAM, 2 CPUs): Adapter 2 is the only network adapter and is Host-only; USB controller xHCI with 0 active device filters. [Source: baseline_configuration_of_the_windows_11.png]

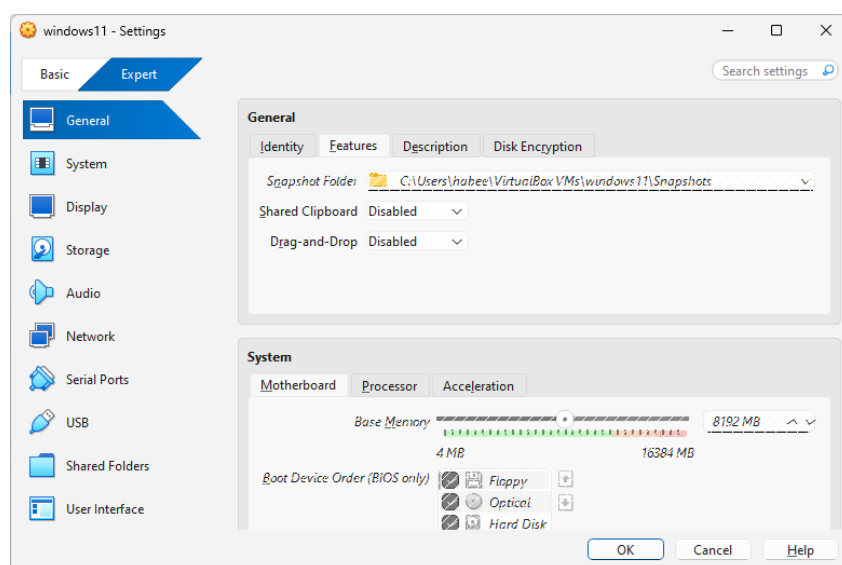


Figure 3. VirtualBox Settings > General > Features for windows11: Shared Clipboard and Drag-and-Drop are both Disabled. [Source: screenshot_2.png]

All three captures agree: the only network path is the Host-only adapter, and clipboard, drag-and-drop and shared folders are off. The first VBoxManage attempt returned a "FindMachine" error (visible at the top of the first capture) and was re-run with the correct VM name; this is recorded in Appendix C.

2.2 Environment record

VM name / guest hostname	windows11 / flare
Guest operating system	Windows 11 64-bit, Microsoft Windows [Version 10.0.22621.1555] (FLARE-VM)
Resources	8192 MB RAM, 2 processors, 97.01 GB virtual disk
Date and time recorded	Thursday, October 1, 2026 4:55:54 PM (VM clock)
Working folders	C:\L\o (original), C:\L\u (unpacked copy), C:\L\e (evidence)
Time references	VM clock for guest events; host clock for VirtualBox snapshot times. No capture shows the two clocks side by side, so the exact offset was not recorded (see 6.5).

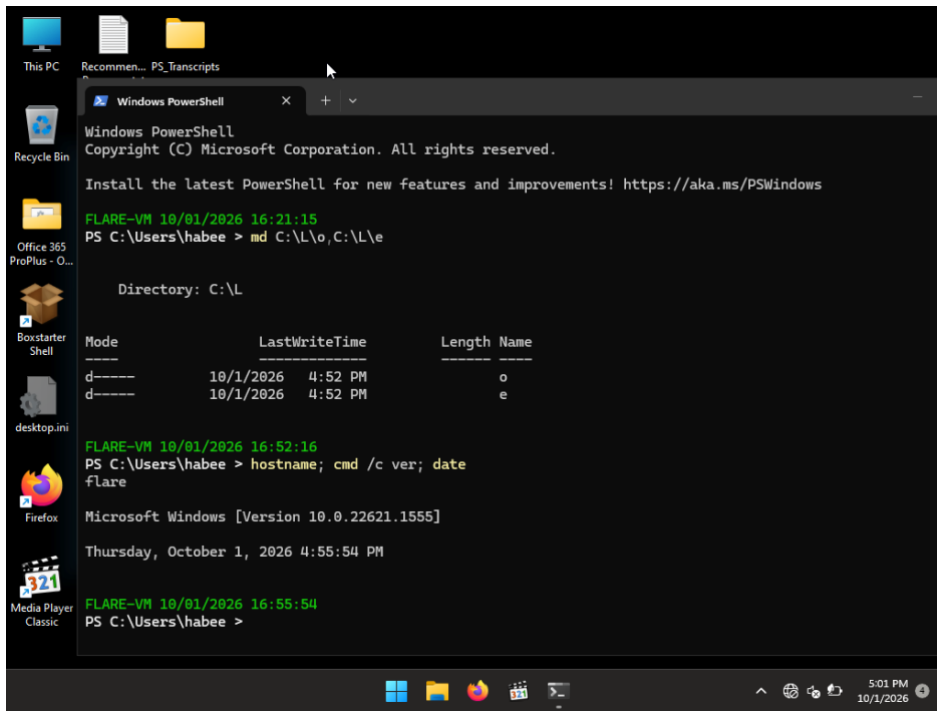


Figure 5. Guest environment record: folders C:\L\o and C:\L\e created (LastWriteTime 4:52 PM); hostname "flare"; Microsoft Windows [Version 10.0.22621.1555]; date Thursday, October 1, 2026 4:55:54 PM (VM clock). [Source: screenshot_6.png]

2.3 Snapshots

Snapshot	Taken (host clock)	Description (as entered)	Role
win11	9/29/2026 7:59 PM	update disabled	Base FLARE-VM state
Snapshot 2	9/30/2026 12:02 AM	antivirus removed and deactivated	Intermediate preparation state
Lab02_PreExecution_Baseline	10/1/2026 3:08 PM	Flare vm installed	Clean baseline; restore target at the end
Lab02_PreExecution_ToolsReady	10/1/2026 9:04 PM	Monitoring tools running, sample not yet executed	Tools-ready state (timing caveat in 6.5)
Lab02_PostExecution_Evidence	10/1/2026 10:06 PM	After run: Procmon, FakeNet, Regshot saved in C:\L\e	Evidence preserved after the runs

The clean baseline was taken at 3:08 PM, before the C:\L folders were created (4:52 PM) and before the sample was extracted (present in C:\L\o at 5:15 PM). The time the archive was downloaded into the VM was not recorded, so the report does not claim the snapshot preceded the download itself.

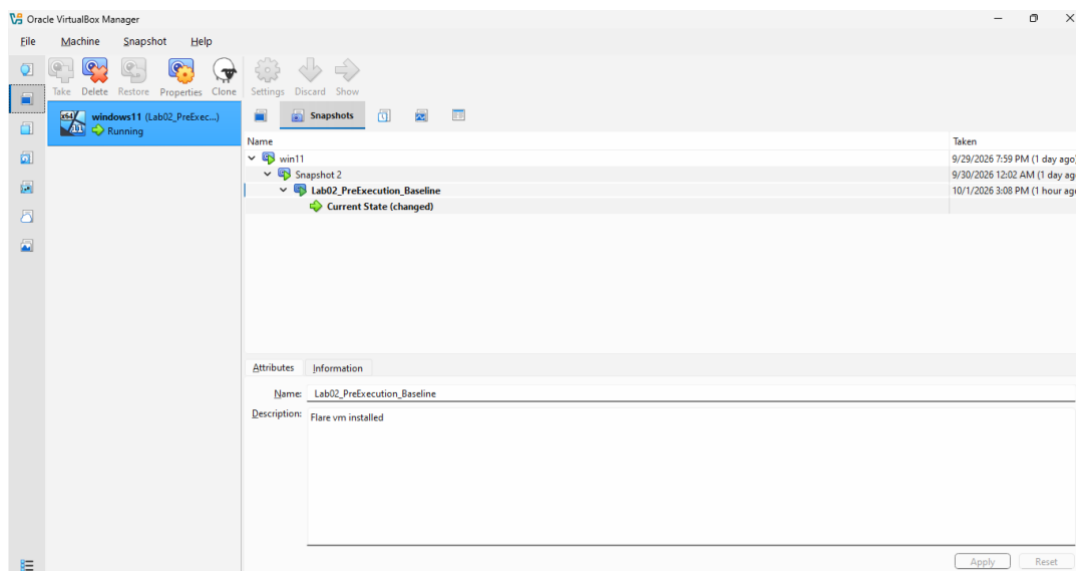


Figure 4. VirtualBox Snapshots panel: Lab02_PreExecution_Baseline, taken 10/1/2026 3:08 PM (host clock), description "Flare vm installed". The VM was running; the current state sits below the snapshot. [Source: Lab02_PreExecution_Baseline.png]

2.4 Evidence tools

Tool	Version / location	Purpose
VirtualBox, VBoxManage (host)	Host	Isolation proof and snapshots
Windows PowerShell	Guest	Baselines, hashing, execution, comparisons
UPX	5.2.0 (Jun 8th 2026), C:\ProgramData\chocolatey\bin	Packing test and approved unpack
Detect It Easy (die.exe, diec.exe)	v3.10, C:\Tools\die	Packer detection, entropy, sections
pestudio	9.61	Sections, entropy, imports, indicators
PE Detective	Deep scan	Independent packer-signature check
Process Monitor, Process Explorer	Sysinternals	Process and file monitoring during execution
Regshot	1.9.1 x64 Unicode	Registry before and after comparison
Autoruns	Sysinternals	Autostart baseline and comparison
FakeNet-NG	3.5, C:\tools\fakenet\fakenet3.5	Simulated network services and connection logging
Wireshark	Capture file packets_20261001_195556.pcap	Packet capture

PE-bear was not installed on this FLARE-VM build and PE Carve, although present, was deliberately not used because it extracts embedded files and could alter the evidence set.

2.5 Baselines before analysis

Before the sample was handled, the following baselines were saved to C:\L\e: network connections (netstat -ano to b_net.txt), the HKCU and HKLM Run keys (b_u.txt and b_m.txt) and scheduled tasks (schtasks to b_t.txt). The process list was displayed on screen but the b_ps.txt file was not saved, so there is no saved process baseline for comparison. An Autoruns baseline (autoruns_before.arn) was saved at 8:27 PM, immediately before the execution window (Section 4).

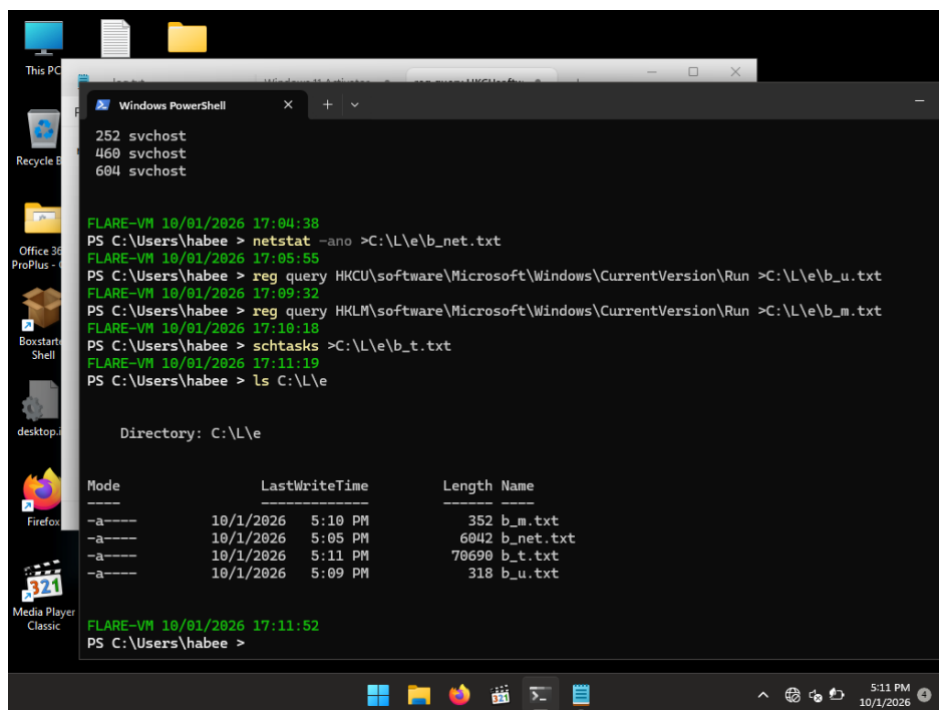


Figure 6. Baseline captures before analysis: b_net.txt (5:05 PM), b_u.txt (5:09 PM), b_m.txt (5:10 PM) and b_t.txt (5:11 PM) saved to C:\L\e by netstat, reg query and schtasks. The process list at the top of the window was shown on screen only; no b_ps.txt file was saved. [Source: screenshot_7.png]

2.6 Original sample and hash

The sample was extracted into C:\L\o without being executed. It is a 43,520-byte file whose file-system modified date is 2/7/2018 12:55 AM. Its SHA-256, recorded before unpacking or execution, is:

F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDDFF36101443975E5B5A

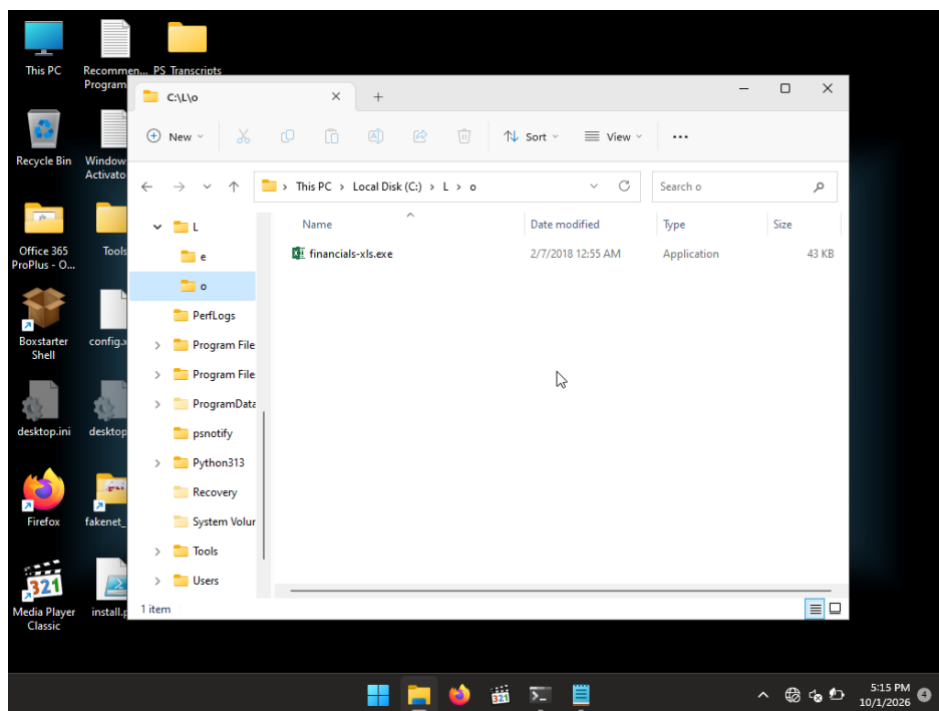


Figure 7. The sample extracted into C:\L\o: financials-xls.exe, 43 KB, file-system date modified 2/7/2018 12:55 AM (VM clock 5:15 PM). [Source: Extracted_malware_to_the_original_folder.png]

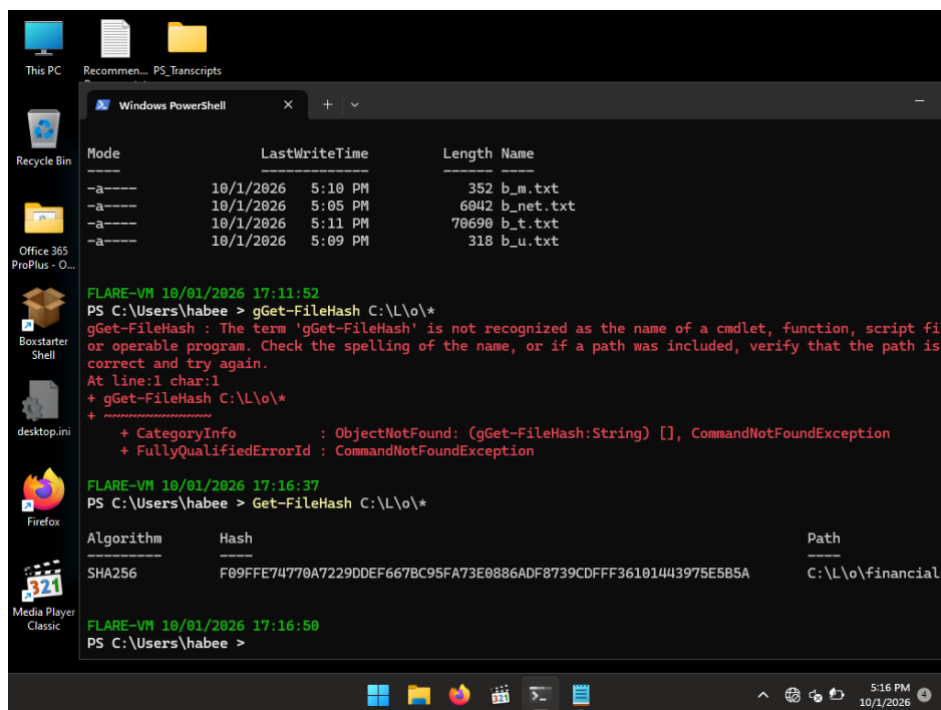


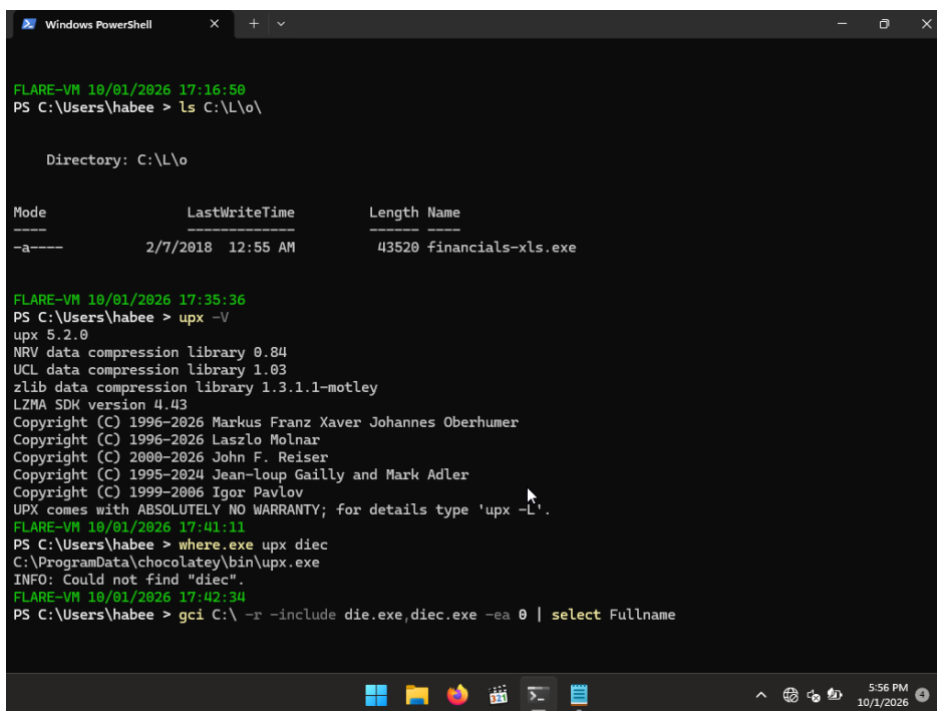
Figure 8. SHA-256 of the original sample (VM prompt time 17:16:37): F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A. The first attempt, mistyped as gGet-FileHash, failed and was repeated. [Source: Original_malware_file_hashed.png]

3 Packing validation and approved unpacking (Section B)

The course states the sample is UPX-packed. This section shows how that was confirmed with independent evidence, how the file was unpacked safely under a new name, and how the two files compare. Nothing in this section executed the sample: every tool only read the file, and UPX was used with -t (test in memory) and -d -o (decompress to a new file).

3.1 Tool inventory

upx -V confirmed UPX 5.2.0. The Detect It Easy command-line tool (diec) was not on the PATH, so where.exe could not find it; a file search located both die.exe and diec.exe in C:\Tools\die, and they were called by full path.



```
FLARE-VM 10/01/2026 17:16:50
PS C:\Users\habee > ls C:\L\o\

Directory: C:\L\o

Mode                LastWriteTime         Length Name
----                -
-a-----          2/7/2018  12:55 AM         43520 financials-xls.exe

FLARE-VM 10/01/2026 17:35:36
PS C:\Users\habee > upx -V
upx 5.2.0
NRV data compression library 0.84
UCL data compression library 1.03
zlib data compression library 1.3.1.1-motley
LZMA SDK version 4.43
Copyright (C) 1996-2026 Markus Franz Xavier Johannes Oberhumer
Copyright (C) 1996-2026 Laszlo Molnar
Copyright (C) 2000-2026 John F. Reiser
Copyright (C) 1995-2024 Jean-loup Gailly and Mark Adler
Copyright (C) 1999-2006 Igor Pavlov
UPX comes with ABSOLUTELY NO WARRANTY; for details type 'upx -L'.
FLARE-VM 10/01/2026 17:41:11
PS C:\Users\habee > where.exe upx diec
C:\ProgramData\chocolatey\bin\upx.exe
INFO: Could not find "diec".
FLARE-VM 10/01/2026 17:42:34
PS C:\Users\habee > gci C:\ -r -include die.exe,diec.exe -ea 0 | select Fullname
```

Figure 9. Tool inventory: upx -V reports UPX 5.2.0; where.exe finds upx under C:\ProgramData\chocolatey\bin but not diec (prompt times 17:35:36 to 17:42:34). [Source: screenshot_9.png]

```

Windows PowerShell

Mode                LastWriteTime         Length Name
-----
-a-----          2/7/2018  12:55 AM           43520 financials-xls.exe

FLARE-VM 10/01/2026 17:35:36
PS C:\Users\habee > upx -V
upx 5.2.0
NRV data compression library 0.84
UCL data compression library 1.03
zlib data compression library 1.3.1.1-motley
LZMA SDK version 4.43
Copyright (C) 1996-2026 Markus Franz Xavier Johannes Oberhumer
Copyright (C) 1996-2026 Laszlo Molnar
Copyright (C) 2000-2026 John F. Reiser
Copyright (C) 1995-2024 Jean-loup Gailly and Mark Adler
Copyright (C) 1999-2006 Igor Pavlov
UPX comes with ABSOLUTELY NO WARRANTY; for details type 'upx -L'.
FLARE-VM 10/01/2026 17:41:11
PS C:\Users\habee > where.exe upx diec
C:\ProgramData\chocolatey\bin\upx.exe
INFO: Could not find "diec".
FLARE-VM 10/01/2026 17:42:34
PS C:\Users\habee > gci C:\ -r -include die.exe,diec.exe -ea 0 | select Fullname

FullName
-----
C:\Tools\die\die.exe
C:\Tools\die\diec.exe

FLARE-VM 10/01/2026 17:59:13
PS C:\Users\habee >

```

Figure 10. Search for the DIE files: C:\Tools\die\die.exe and C:\Tools\die\diec.exe (prompt time 17:59:13). [Source: screenshot_10.png]

3.2 Packing indicators

Detect It Easy

DIE identified the packer by signature as **UPX(3.92)[NRV,best]** on the command line and in the GUI, and the GUI also raised a heuristic detection of compressed or packed data based on the entry point, UPX-like imports and section characteristics. The command-line heuristic scan is disabled by default, so the command-line result is signature-based.

```

Windows PowerShell

Copyright (C) 1996-2026 Laszlo Molnar
Copyright (C) 2000-2026 John F. Reiser
Copyright (C) 1995-2024 Jean-loup Gailly and Mark Adler
Copyright (C) 1999-2006 Igor Pavlov
UPX comes with ABSOLUTELY NO WARRANTY; for details type 'upx -L'.
FLARE-VM 10/01/2026 17:41:11
PS C:\Users\habee > where.exe upx diec
C:\ProgramData\chocolatey\bin\upx.exe
INFO: Could not find "diec".
FLARE-VM 10/01/2026 17:42:34
PS C:\Users\habee > gci C:\ -r -include die.exe,diec.exe -ea 0 | select Fullname

FullName
-----
C:\Tools\die\die.exe
C:\Tools\die\diec.exe

FLARE-VM 10/01/2026 17:59:13
PS C:\Users\habee > & "C:\Tools\die\diec.exe" ^C
FLARE-VM 10/01/2026 18:07:27
PS C:\Users\habee > ^C
FLARE-VM 10/01/2026 18:07:38
PS C:\Users\habee > & "C:\Tools\die\diec.exe" C:\L\o\financials-xls.exe
Cannot find: C:\L\o\financials-xls.exe
FLARE-VM 10/01/2026 18:09:56
PS C:\Users\habee > & "C:\Tools\die\diec.exe" C:\L\o\financials-xls.exe
[!] Heuristic scan is disabled. Use '--heuristicscan' to enable
PE32
  Linker: Microsoft Linker(5.12.8034)
  Compiler: Microsoft Visual C/C++(12.00.8168)[C++]
  Packer: UPX(3.92)[NRV,best]

FLARE-VM 10/01/2026 18:11:07
PS C:\Users\habee >

```

Figure 11. DIE command line (diec.exe) on the original. The first attempt contained a typo (financials-xls.exe) and returned "Cannot find"; the corrected command (18:09:56) reports PE32, Microsoft Linker 5.12.8034, Microsoft Visual C/C++ 12.00.8168 and Packer: UPX(3.92)[NRV,best]. The command-line heuristic scan was disabled by default. [Source: screenshot_11.png]

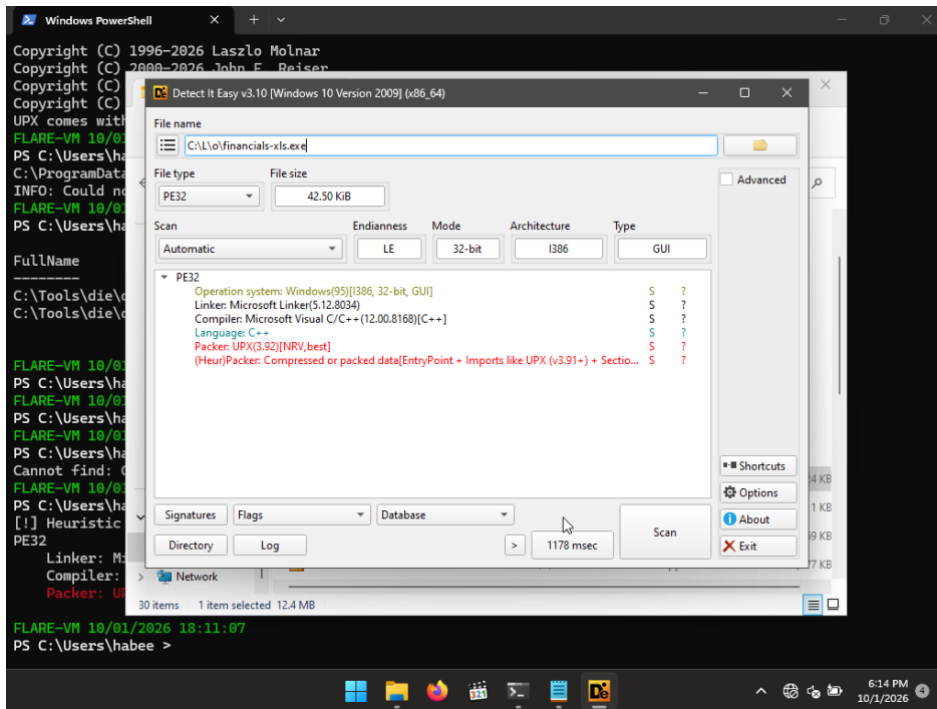


Figure 12. Detect It Easy v3.10 GUI on the original (taskbar 6:14 PM): PE32, 42.50 KiB, Packer: UPX(3.92)[NRV,best], plus a heuristic detection "Compressed or packed data [EntryPoint + Imports like UPX (v3.91+) + Section...]". [Source: screenshot_12.png]

The entropy view shows why per-section figures matter. The **whole-file** entropy is 5.565 and DIE's summary bar reads "not packed 69%", because the low-entropy resource section (.rsrc, 3.470) dilutes the average. The per-region figures tell the real story: the PE header region is 6.985 and the **UPX1 section is 7.844**, both flagged "packed", and the graph holds a plateau near 7.5 across the UPX1 byte range before dropping sharply at the start of the resources. Values above about 7 are typical of compressed data.

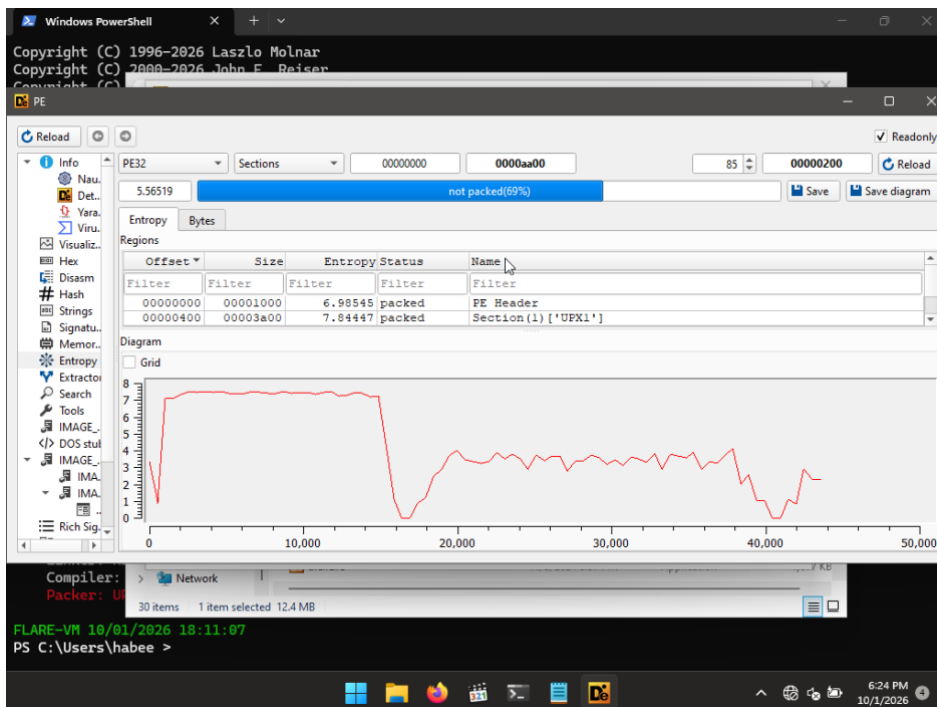


Figure 13. DIE entropy view for the original (6:24 PM): whole-file entropy 5.56519 (status bar "not packed 69%"); the region table lists PE Header 6.98545 and Section(1) [UPX1] 7.84447, both marked "packed". The graph shows a sustained plateau near 7.5 across the UPX1 range. [Source: screenshot_13.png]

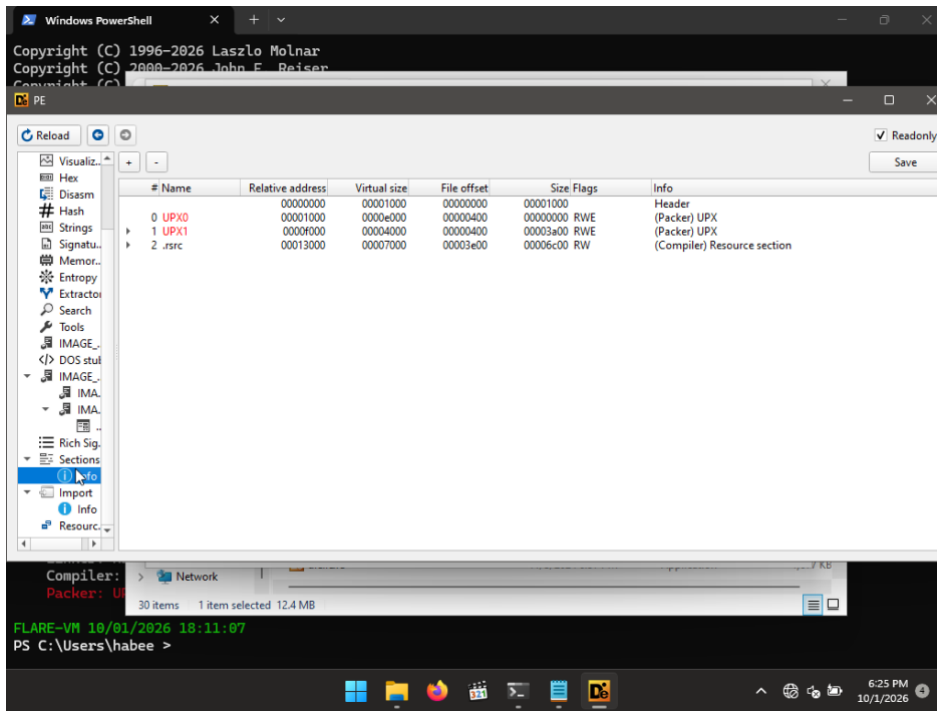


Figure 14. DIE section table for the original (6:25 PM): UPX0 (RVA 0x1000, virtual size 0xE000, file size 0, RWE), UPX1 (RVA 0xF000, virtual 0x4000, file offset 0x400, size 0x3A00, RWE) and .rsrc (RVA 0x13000, size 0x6C00, RW). [Source: screenshot_14.png]

pestudio

pestudio gives the same structural picture. The file has only three sections: **UPX0** (raw size 0 bytes but virtual size 57,344 bytes, the empty space into which the stub decompresses), **UPX1** (14,848 raw bytes, entropy 7.844, 34.12% of the file) and **.rsrc** (27,648 bytes, entropy 3.470). The entry point (0x00012790) lies inside UPX1, where the unpacking stub lives, and pestudio flags UPX0 and UPX1 as writable, executable and self-modifying. Only 10 imports are visible (including LoadLibraryA, GetProcAddress and VirtualProtect), which is typical of a stub that resolves the real imports at run time.

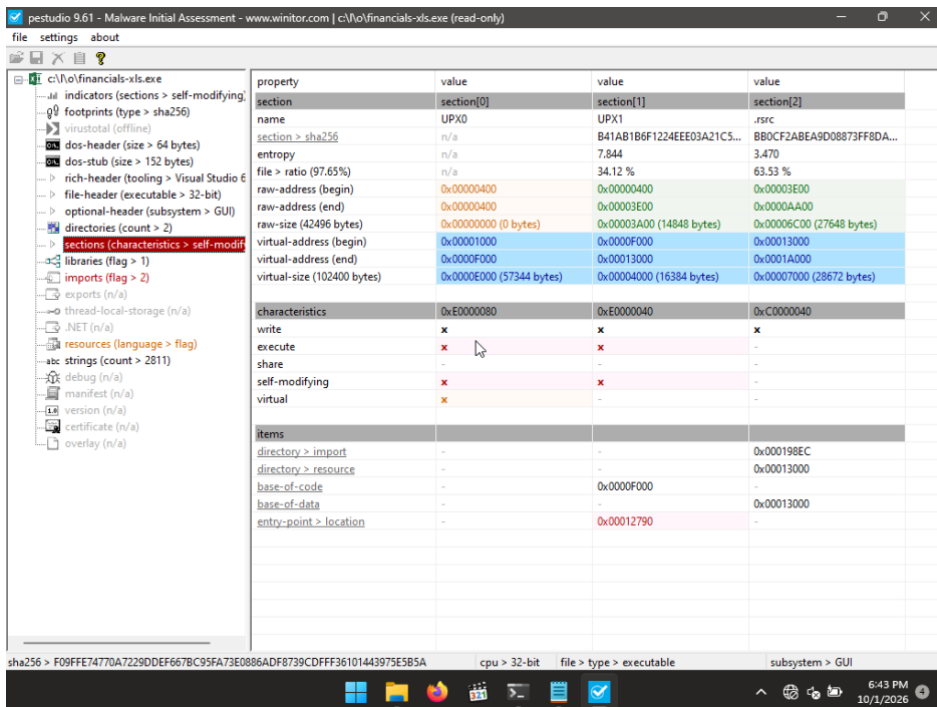


Figure 15. pestudio 9.61 sections view for the original (6:43 PM): UPX0 raw size 0 bytes with virtual size 57,344 bytes; UPX1 entropy 7.844, raw size 14,848 bytes; .rsrc entropy 3.470, raw size 27,648 bytes; entry point 0x00012790 in UPX1; UPX0 and UPX1 flagged self-modifying. [Source: screenshot_15.png]

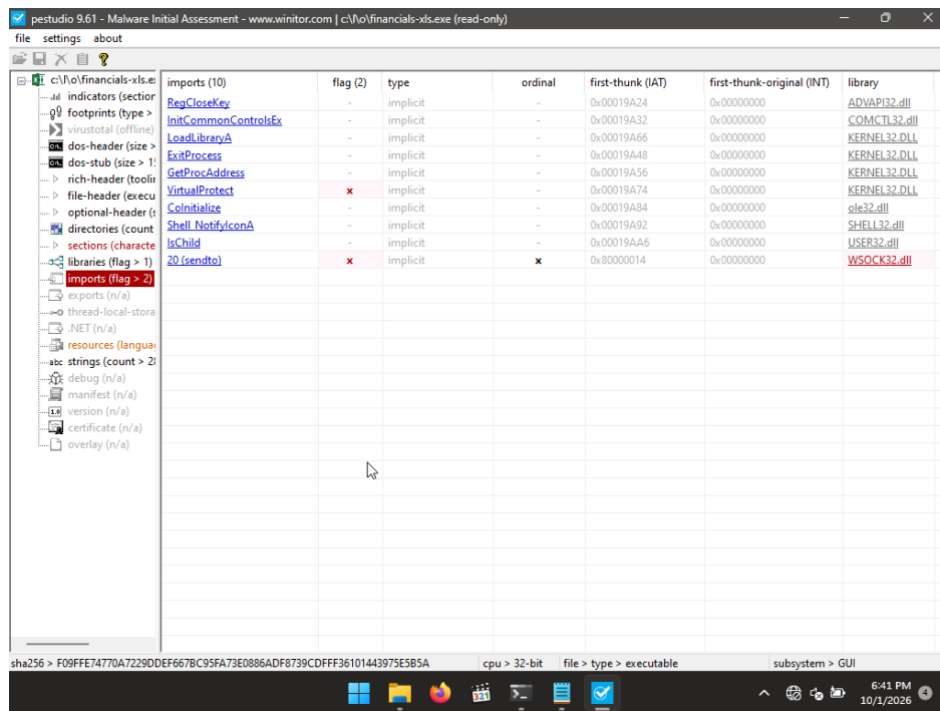


Figure 16. pestudio imports view for the original (6:41 PM): 10 imports, 2 flagged (VirtualProtect and ordinal 20, sendto, from WSOCK32.dll). LoadLibraryA and GetProcAddress are among the few resolved imports. [Source: screenshot_16.png]

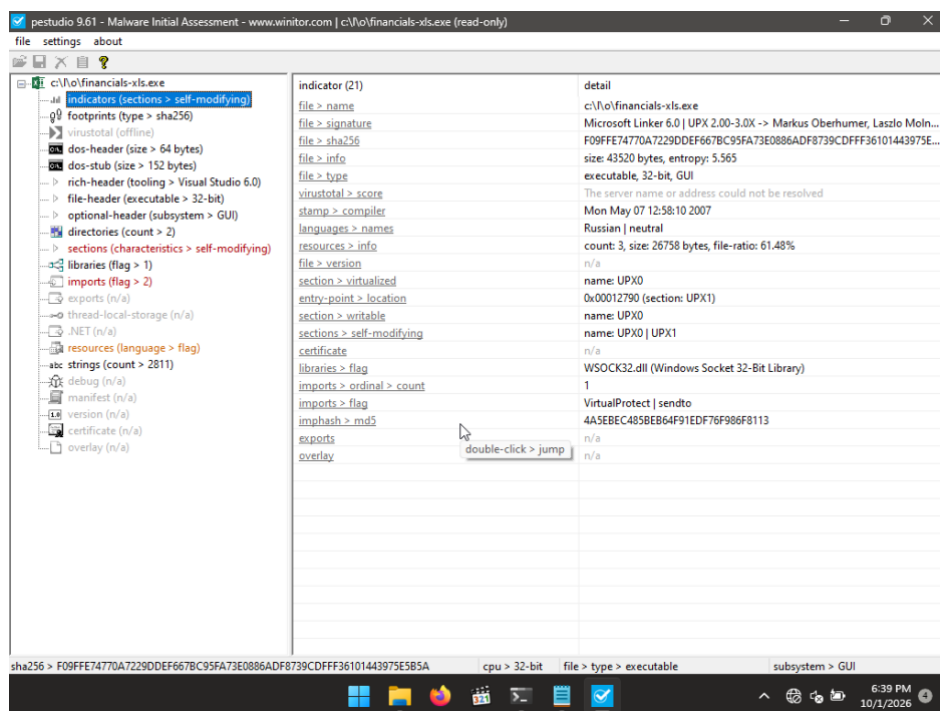


Figure 17. pestudio indicators view for the original (6:39 PM): signature "UPX 2.00-3.0X", entropy 5.565, size 43,520 bytes, compiler stamp Mon May 07 12:58:10 2007, language Russian | neutral, imphash 4A5EBEC485BEB64F91EDF76F986F8113; the VirusTotal lookup is offline. [Source: screenshot_17.png]

PE Detective

PE Detective's deep scan matched three UPX signature families, with "UPX 2.90 [LZMA] (Delphi stub)" as the best match.

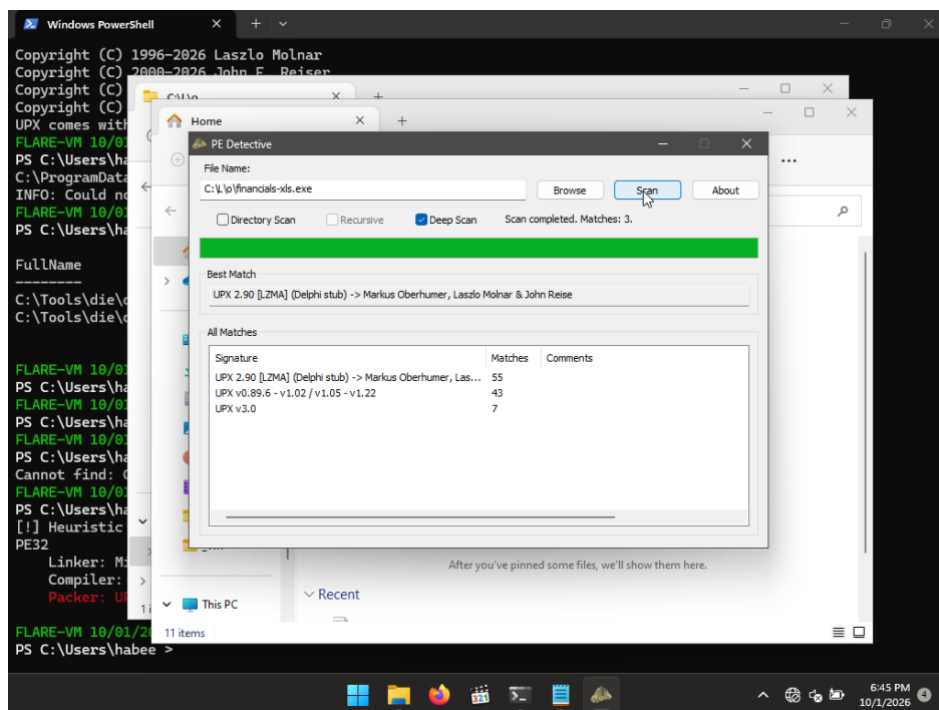


Figure 18. PE Detective deep scan of the original (6:45 PM): best match "UPX 2.90 [LZMA] (Delphi stub)"; three UPX signature families matched (55, 43 and 7 matches). [Source: screenshot_18.png]

UPX integrity test

UPX itself recognised the file: `upx -t` reported **[OK]** and `upx -l` listed 57,344 bytes unpacked against 43,520 bytes packed (75.89%). The packed size equals the 42.50 KiB reported by DIE.

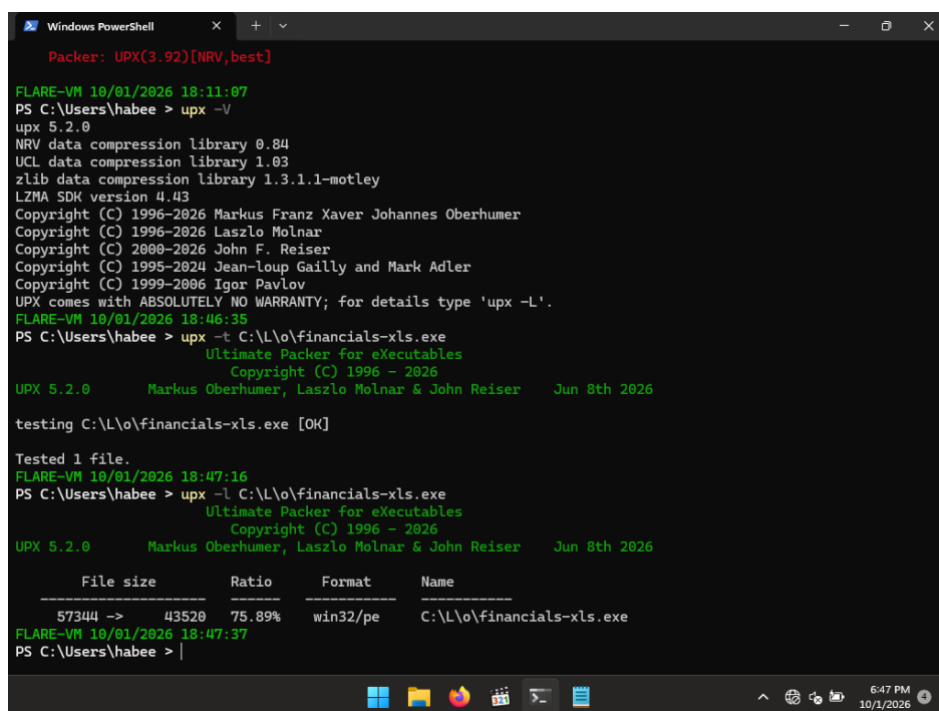


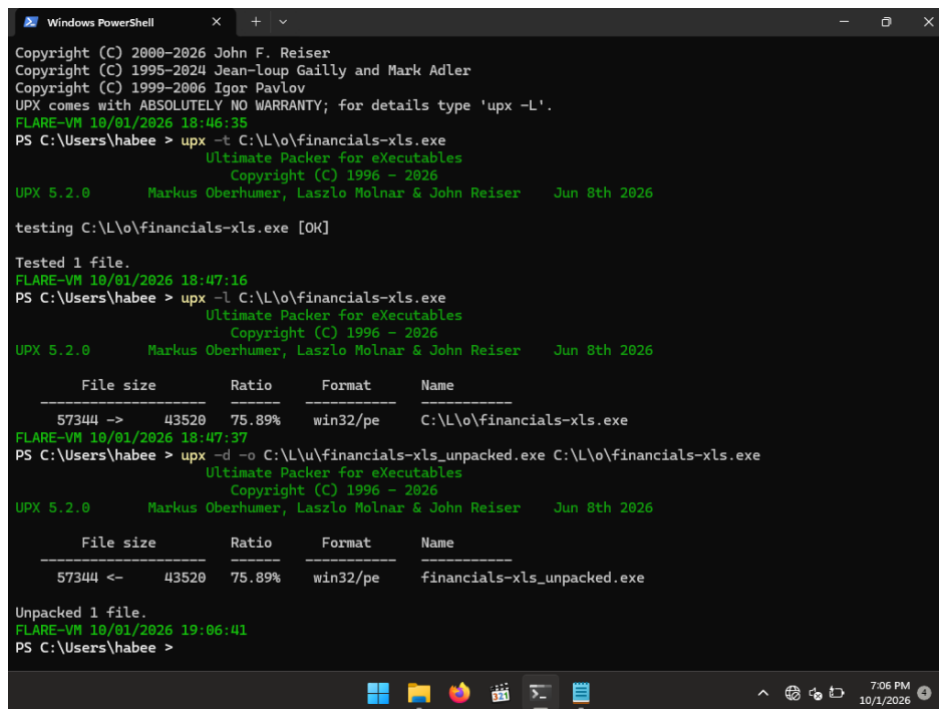
Figure 19. UPX 5.2.0 integrity test and listing on the original (prompt times 18:46:35 and 18:47:16; taskbar 6:47 PM): "testing C:\L\o\financials-xls.exe [OK]"; listing 57344 -> 43520, ratio 75.89%, format win32/pe. [Source: screenshot_19.png]

Assessment. Four independent methods (signature match, section and entropy structure, UPX's own test and, later, a successful unpack) all support the packing conclusion. Confidence is **high**. The tools disagree on the exact UPX version (DIE 3.92, pestudio "UPX 2.00-3.0X", PE Detective "UPX 2.90 [LZMA]"), so the version is treated as unresolved and only "UPX" is asserted. The unpacker used was 5.2.0.

3.3 Approved unpacking and hashes

The sample was unpacked inside the VM with the approved workflow, writing to a new filename so the original stayed untouched:

```
upx -d -o C:\L\u\financials-xls_unpacked.exe C:\L\o\financials-xls.exe
```



```
Windows PowerShell
Copyright (C) 2009-2026 John F. Reiser
Copyright (C) 1995-2024 Jean-Loup Gailly and Mark Adler
Copyright (C) 1999-2006 Igor Pavlov
UPX comes with ABSOLUTELY NO WARRANTY; for details type 'upx -L'.
FLARE-VM 10/01/2026 18:46:35
PS C:\Users\habee > upx -t C:\L\o\financials-xls.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

testing C:\L\o\financials-xls.exe [OK]

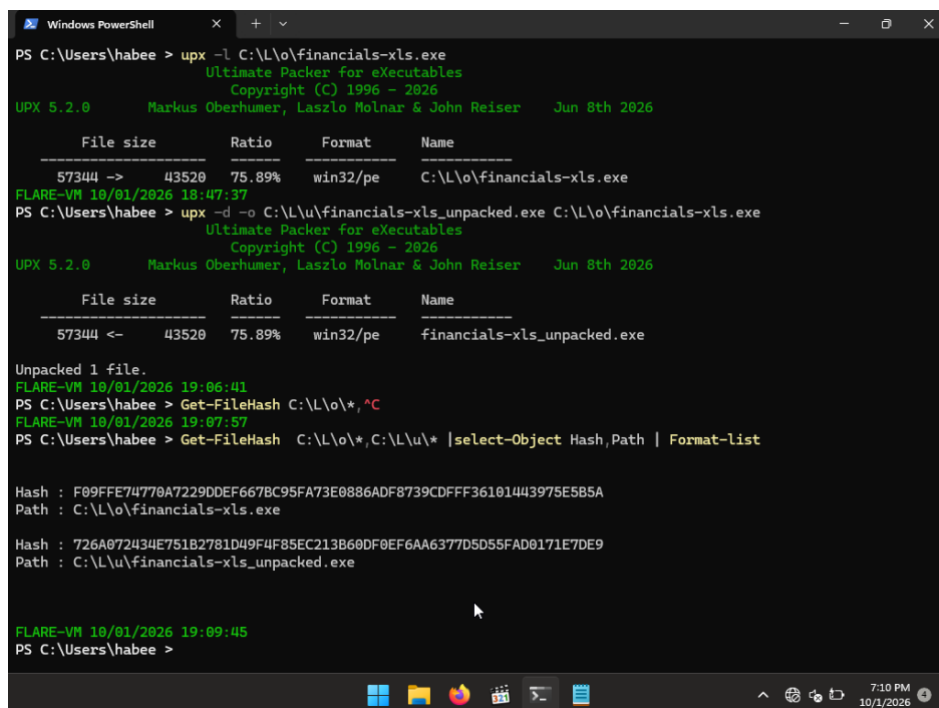
Tested 1 file.
FLARE-VM 10/01/2026 18:47:16
PS C:\Users\habee > upx -l C:\L\o\financials-xls.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

  File size  Ratio  Format  Name
-----
573444 -> 43520 75.89% win32/pe C:\L\o\financials-xls.exe
FLARE-VM 10/01/2026 18:47:37
PS C:\Users\habee > upx -d -o C:\L\u\financials-xls_unpacked.exe C:\L\o\financials-xls.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

  File size  Ratio  Format  Name
-----
573444 <- 43520 75.89% win32/pe financials-xls_unpacked.exe

Unpacked 1 file.
FLARE-VM 10/01/2026 19:06:41
PS C:\Users\habee >
```

Figure 20. Approved unpack, with the test and listing above it: `upx -d -o C:\L\u\financials-xls_unpacked.exe C:\L\o\financials-xls.exe` reports "Unpacked 1 file" (taskbar 7:06 PM). [Source: screenshot_20.png]



```
Windows PowerShell
PS C:\Users\habee > upx -l C:\L\o\financials-xls.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

  File size  Ratio  Format  Name
-----
573444 -> 43520 75.89% win32/pe C:\L\o\financials-xls.exe
FLARE-VM 10/01/2026 18:47:37
PS C:\Users\habee > upx -d -o C:\L\u\financials-xls_unpacked.exe C:\L\o\financials-xls.exe
      Ultimate Packer for eXecutables
      Copyright (C) 1996 - 2026
UPX 5.2.0      Markus Oberhumer, Laszlo Molnar & John Reiser      Jun 8th 2026

  File size  Ratio  Format  Name
-----
573444 <- 43520 75.89% win32/pe financials-xls_unpacked.exe

Unpacked 1 file.
FLARE-VM 10/01/2026 19:06:41
PS C:\Users\habee > Get-FileHash C:\L\o\*,*C
FLARE-VM 10/01/2026 19:07:57
PS C:\Users\habee > Get-FileHash C:\L\o\*,C:\L\u\* |select-Object Hash,Path | Format-List

Hash : F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A
Path : C:\L\o\financials-xls.exe

Hash : 726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9
Path : C:\L\u\financials-xls_unpacked.exe

FLARE-VM 10/01/2026 19:09:45
PS C:\Users\habee >
```

Figure 21. Hashes of both files after unpacking (command at prompt 19:07:57): original F09FFE74...5E5B5A (unchanged) and unpacked 726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9. A first `Get-FileHash` was interrupted with `Ctrl+C` at 19:06:41. [Source: screenshot_21.png]

File	Path	SHA-256
Original	C:\L\o\financials-xls.exe	F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A

File	Path	SHA-256
Unpacked	C:\L\u\financials-xls_unpacked.exe	726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9

The original's hash is identical in Section A, after unpacking and after the execution window (Section 7), so unpacking and execution did not alter it. The relationship between the two files is established by the unpack command and its output (UPX writes the new file from the named original) and is recorded in the evidence log.

3.4 Original versus unpacked

Characteristic	Original (packed)	Unpacked
Size	43,520 bytes	57,344 bytes
SHA-256	F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A	726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9
DIE detection	Packer: UPX(3.92)[NRV,best]; heuristic packed-data detection (GUI)	No packer reported (command line)
Sections	UPX0 (raw 0), UPX1, .rsrc	.text, .data, .rsrc
Whole-file entropy (DIE)	5.565	5.272
Key section entropy	UPX1 7.844 ("packed"); PE header 6.985	.text 6.388 ("not packed"); PE header 2.049
Imports (pestudio)	10 (2 flagged: VirtualProtect, sendto)	85 (20 flagged)
Import visibility	Only the stub's imports plus a few others	Registry (RegCreateKeyExA, RegSetValueExA, RegDeleteValueA), shell notification (Shell_NotifyIconA), window (FindWindowExA, GetDesktopWindow) and socket (connect, recvfrom) functions

```

Windows PowerShell
57344 <- 43520 75.89% win32/pe financials-xls_unpacked.exe

Unpacked 1 file.
FLARE-VM 10/01/2026 19:06:41
PS C:\Users\habee > Get-FileHash C:\L\o\*,*C
FLARE-VM 10/01/2026 19:07:57
PS C:\Users\habee > Get-FileHash C:\L\o\*,C:\L\u\* |select-Object Hash,Path | Format-List

Hash : F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A
Path : C:\L\o\financials-xls.exe
Hash : 726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9
Path : C:\L\u\financials-xls_unpacked.exe

FLARE-VM 10/01/2026 19:09:45
PS C:\Users\habee > ls C:\L\o,C:\L\u | Select-Object Name,Length

Name                               Length
----                               -
financials-xls.exe                  43520
financials-xls_unpacked.exe         57344

FLARE-VM 10/01/2026 19:13:13
PS C:\Users\habee > & "C:\Tools\die\die.exe" C:\L\u\financials-xls_unpacked.exe
[!] Heuristic scan is disabled. Use '--heuristicscan' to enable
PE32
  Linker: Microsoft Linker(5.12.8034)
  Compiler: Microsoft Visual C/C++(12.00.8168)[C++]

FLARE-VM 10/01/2026 19:17:22
PS C:\Users\habee >

```

Figure 22. File sizes (original 43,520 bytes; unpacked 57,344 bytes) and DIE command-line scan of the unpacked file (7:17 PM): no packer reported; Microsoft Linker 5.12.8034 and Visual C/C++ 12.00.8168. [Source: screenshot_22.png]

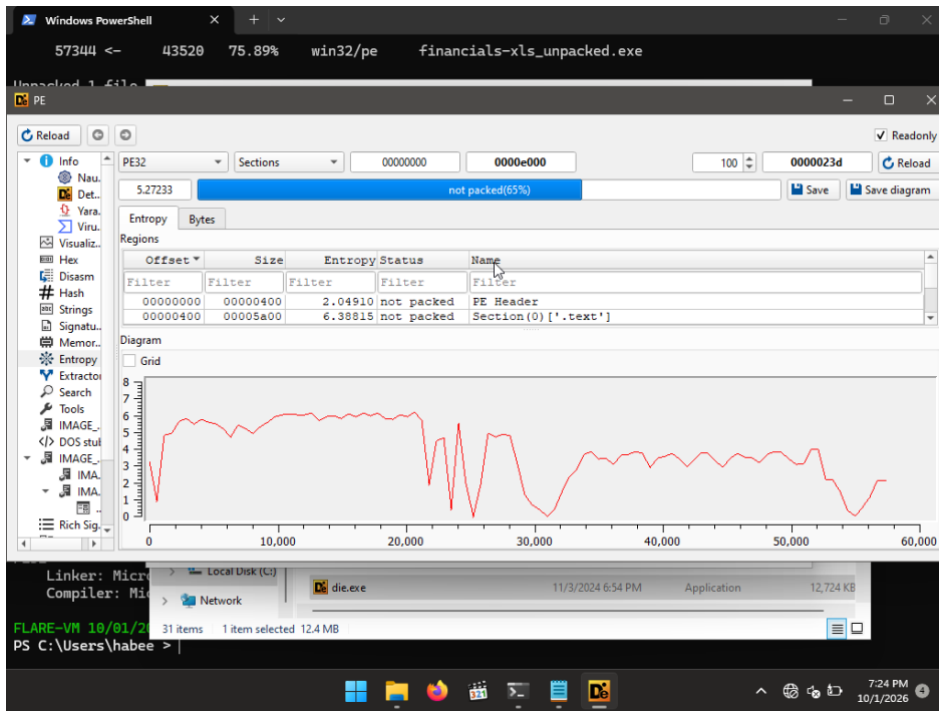


Figure 23. DIE entropy view for the unpacked file (7:24 PM): whole-file entropy 5.27233; PE Header 2.04910 and Section(0) [.text] 6.38815, both "not packed". [Source: screenshot_23.png]

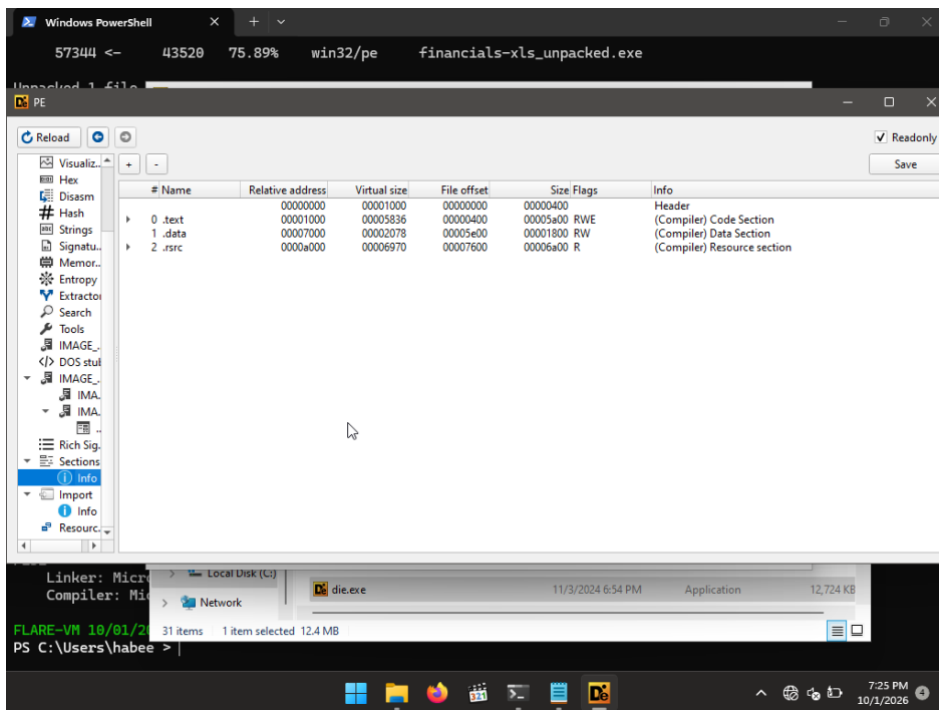


Figure 24. DIE section table for the unpacked file (7:25 PM): .text (virtual 0x5836, size 0x5A00, RWE), .data (0x2078, 0x1800, RW) and .rsrc (0x6970, 0x6A00, R); the UPX0 and UPX1 sections are gone. [Source: screenshot_23b.png]

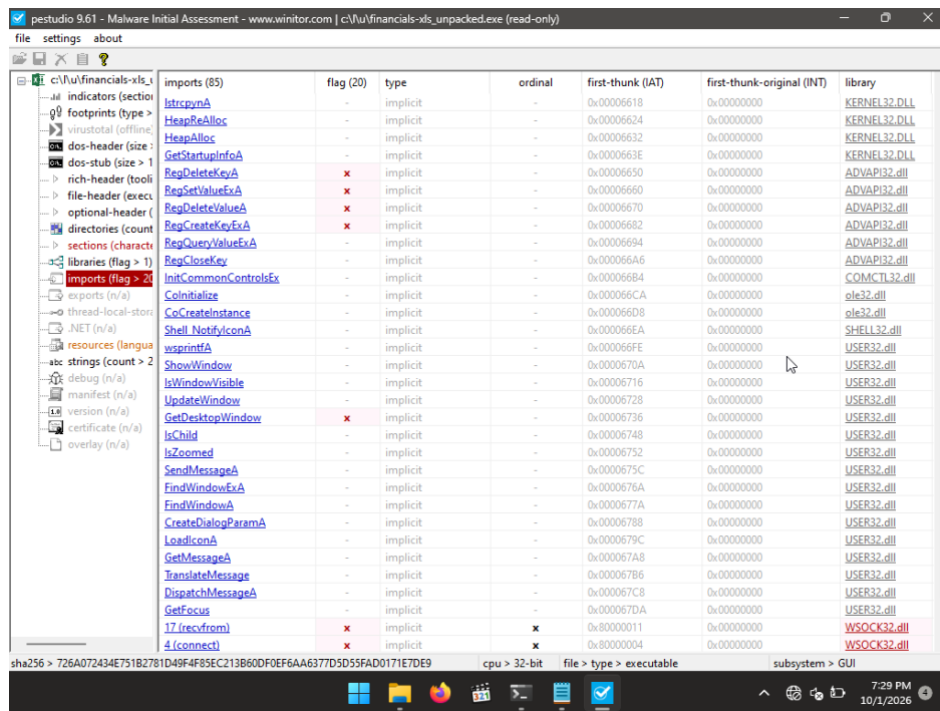


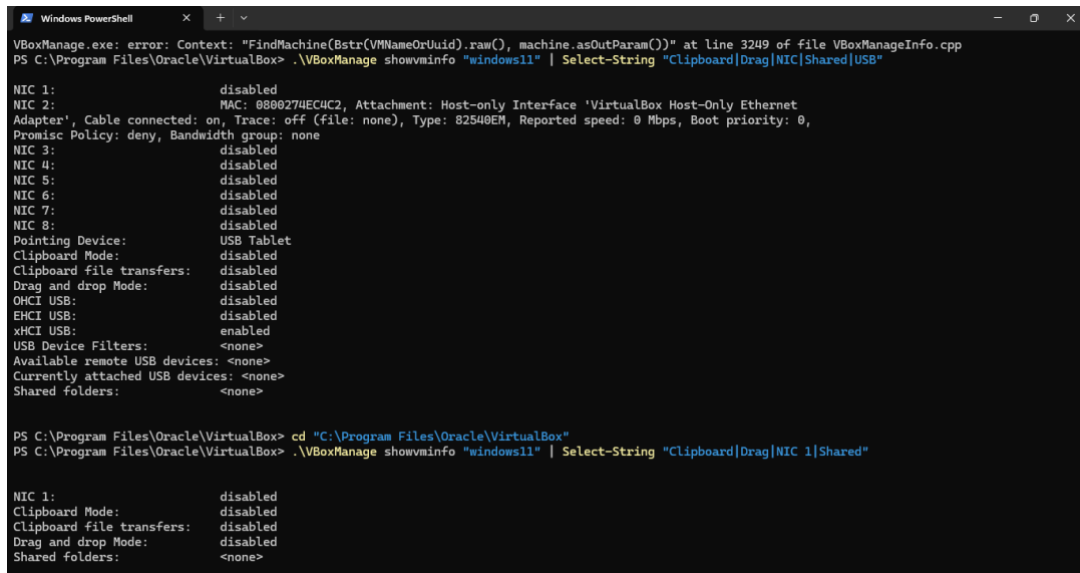
Figure 25. pestudio imports view for the unpacked file (7:29 PM, scrolled): 85 imports, 20 flagged, including RegCreateKeyExA, RegSetValueExA, RegDeleteKeyA, RegDeleteValueA, Shell_NotifyIconA, FindWindowA, FindWindowExA, GetDesktopWindow and WSOCK32 ordinals 17 (recvfrom) and 4 (connect). The status bar SHA-256 matches the unpacked hash. [Source: screenshot_24.png]

After unpacking, the file has a normal compiler-style layout (.text, .data, .rsrc), lower entropy and a much richer import table. These imports show what the program **could** do (modify the registry, post a shell notification, use sockets); they are an inference from static data, not evidence of what it did. Only the runtime observations in Sections 4 and 5 count as observed behaviour.

4 Controlled runtime observation (Section C)

4.1 Preparation and execution window

Before execution the isolation controls were re-verified from the host (NIC 1 disabled, NIC 2 Host-only, clipboard and drag-and-drop disabled, no shared folders) and a tools-ready snapshot was taken. Monitoring was then started in the guest: FakeNet-NG (simulated DNS, HTTP and other services with a Divertor that logs the process behind each connection), a Wireshark capture (file name timestamp 19:55:56), Process Monitor, Process Explorer, Regshot (configured with file scanning off) and Autoruns, whose baseline was saved at 8:27 PM.



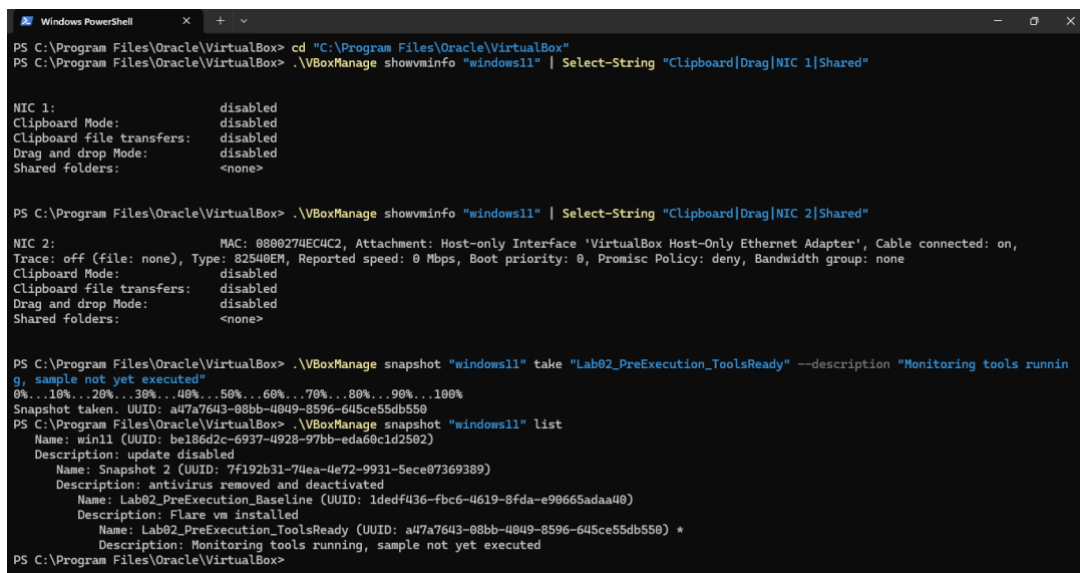
```
Windows PowerShell
VBoxManage.exe: error: Context: "FindMachine(Bstr(VNameOrUuid).raw(), machine.asOutParam())" at line 3249 of file VBoxManageInfo.cpp
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage showvminfo "windows11" | Select-String "Clipboard|Drag|NIC|Shared|USB"

NIC 1: disabled
NIC 2: MAC: 0800274EC4C2, Attachment: Host-only Interface 'VirtualBox Host-Only Ethernet Adapter', Cable connected: on, Trace: off (file: none), Type: 82540EM, Reported speed: 0 Mbps, Boot priority: 0, Promisc Policy: deny, Bandwidth group: none
NIC 3: disabled
NIC 4: disabled
NIC 5: disabled
NIC 6: disabled
NIC 7: disabled
NIC 8: disabled
Pointing Device: USB Tablet
Clipboard Mode: disabled
Clipboard file transfers: disabled
Drag and drop Mode: disabled
OHCI USB: disabled
EHCI USB: disabled
xHCI USB: enabled
USB Device Filters: <none>
Available remote USB devices: <none>
Currently attached USB devices: <none>
Shared folders: <none>

PS C:\Program Files\Oracle\VirtualBox> cd "C:\Program Files\Oracle\VirtualBox"
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage showvminfo "windows11" | Select-String "Clipboard|Drag|NIC 1|Shared"

NIC 1: disabled
Clipboard Mode: disabled
Clipboard file transfers: disabled
Drag and drop Mode: disabled
Shared folders: <none>
```

Figure 26. Isolation controls re-verified from the host with VBoxManage showvminfo before execution: NIC 1 disabled; NIC 2 Host-only; clipboard and drag-and-drop disabled; no shared folders. [Source: Screenshot_29.png]



```
Windows PowerShell
PS C:\Program Files\Oracle\VirtualBox> cd "C:\Program Files\Oracle\VirtualBox"
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage showvminfo "windows11" | Select-String "Clipboard|Drag|NIC 1|Shared"

NIC 1: disabled
Clipboard Mode: disabled
Clipboard file transfers: disabled
Drag and drop Mode: disabled
Shared folders: <none>

PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage showvminfo "windows11" | Select-String "Clipboard|Drag|NIC 2|Shared"

NIC 2: MAC: 0800274EC4C2, Attachment: Host-only Interface 'VirtualBox Host-Only Ethernet Adapter', Cable connected: on, Trace: off (file: none), Type: 82540EM, Reported speed: 0 Mbps, Boot priority: 0, Promisc Policy: deny, Bandwidth group: none
Clipboard Mode: disabled
Clipboard file transfers: disabled
Drag and drop Mode: disabled
Shared folders: <none>

PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage snapshot "windows11" take "Lab02_PreExecution_ToolsReady" --description "Monitoring tools running, sample not yet executed"
0%..10%..20%..30%..40%..50%..60%..70%..80%..90%..100%
Snapshot taken. UUID: a47a7643-08bb-4049-8596-645ce55db550
PS C:\Program Files\Oracle\VirtualBox> .\VBoxManage snapshot "windows11" list
Name: win11 (UUID: be186d2c-6937-4928-97bb-edn60c1d2502)
Description: update disabled
Name: Snapshot 2 (UUID: 7f192b31-74ea-4e72-9931-5ece07369389)
Description: antivirus removed and deactivated
Name: Lab02_PreExecution_Baseline (UUID: 1dedf436-fbc6-4619-8fda-e90665adaa40)
Description: Flare vm installed
Name: Lab02_PreExecution_ToolsReady (UUID: a47a7643-08bb-4049-8596-645ce55db550) *
Description: Monitoring tools running, sample not yet executed
PS C:\Program Files\Oracle\VirtualBox>
```

Figure 27. Snapshot Lab02_PreExecution_ToolsReady taken with VBoxManage ("Monitoring tools running, sample not yet executed") and the resulting list: win11 > Snapshot 2 > Lab02_PreExecution_Baseline > Lab02_PreExecution_ToolsReady. See the timing caveat in Section 6.5. [Source: screenshot_30.png]

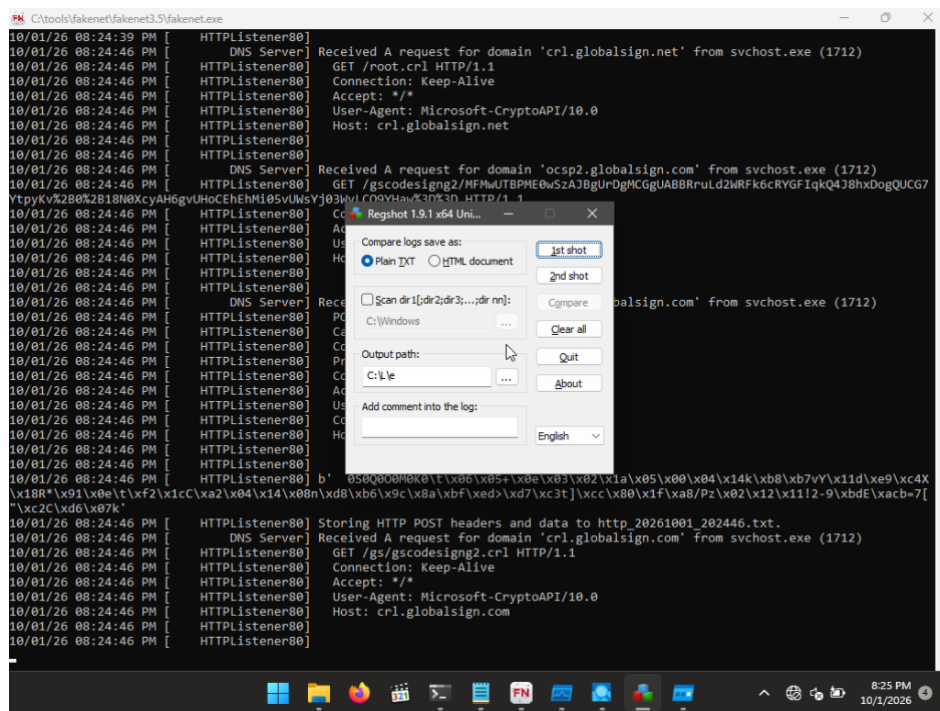


Figure 28. Regshot 1.9.1 x64 Unicode configured before execution (8:25 PM): plain TXT output, output path C:\L\le, "Scan dir" unchecked (file-system scanning off), Compare not yet enabled. The FakeNet-NG console is running behind it. [Source: screenshot_27.png]

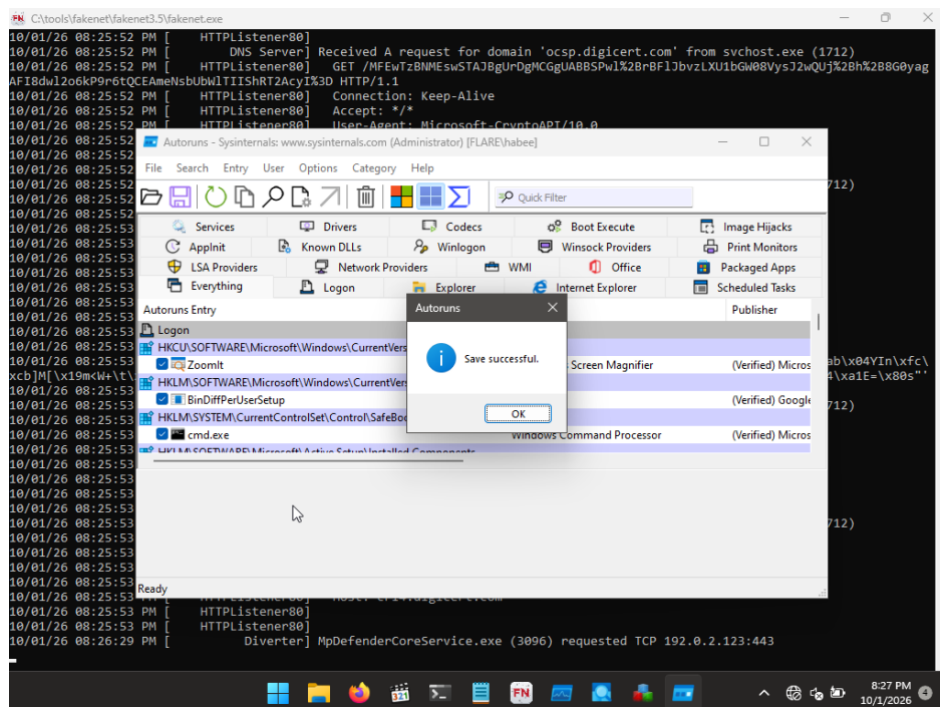


Figure 29. Autoruns baseline saved (8:27 PM): "Save successful" dialog; the file autoruns_before.arn appears in C:\L\le (Figure 39). FakeNet-NG is running behind the window. [Source: screenshot_28.png]

The sample was executed three times inside one window between about 20:49 and 20:57 (VM clock). Only the third run was captured with launch and exit screenshots; the first two runs are known from the FakeNet-NG log, which records the process ID of each connection. Because all three runs fall between the Autoruns baseline (20:27) and the Autoruns comparison (21:09), system changes cannot be attributed to a single run.

4.2 Process observations

Run	PID	Start (VM clock)	Exit	Source
1	1976	about 20:49:00 (first network event)	Not recorded	FakeNet-NG log, Figure 38
2	1600	about 20:53:01 (first network event)	Not recorded	FakeNet-NG log, Figure 38
3	2868	20:57:10	20:57:12 (about 2 s later); HasExited = True	Figure 30, Figure 31, Figure 38

- **Command line:** `Start-Process C:\L\o\financials-xls.exe -PassThru` with no arguments, typed at the prompt at 20:56:56 and returning at 20:57:10 (run 3).
- **Parent:** the launching PowerShell session. Process Monitor shows powershell.exe (PID 2488) opening the sample file at 8:57 PM; the parent-PID column was not captured, so this is an inference.
- **Lifetime:** about two seconds. The process had exited on its own by 20:58:22 (HasExited = True, ExitTime 8:57:12 PM) and no instance was running at 22:08:54. The exit code was not obtained: `$p.ExitCode` was typed in a second PowerShell tab where `$p` was undefined, so it printed nothing.
- **Visible behaviour:** a toast notification titled `financials-xls.exe` reading "Your computer is infected! Windows has detected spyware infection! It is recommended to use special antispyware tools to prevent data loss." The message is cut off on screen.

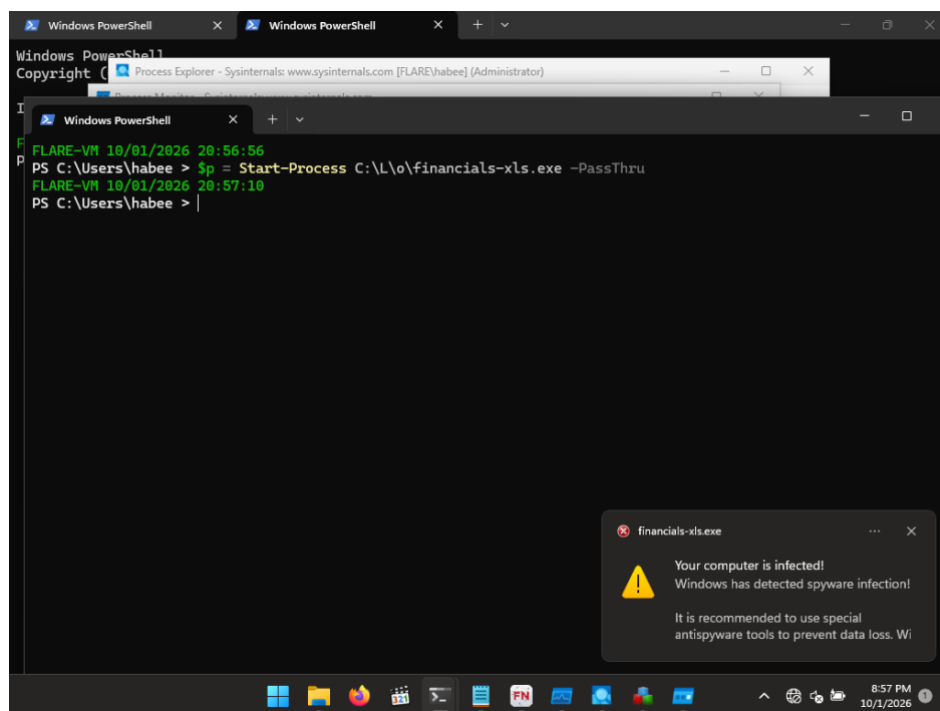


Figure 30. Run 3 launch (VM prompt 20:56:56 to 20:57:10; taskbar 8:57 PM): `$p = Start-Process C:\L\o\financials-xls.exe -PassThru`. A toast titled `financials-xls.exe` appears: "Your computer is infected! Windows has detected spyware infection! It is recommended to use special antispyware tools to prevent data loss." (text cut off on screen). Process Explorer and Process Monitor windows are open behind the shell. [Source: *screenshot_31.png*]

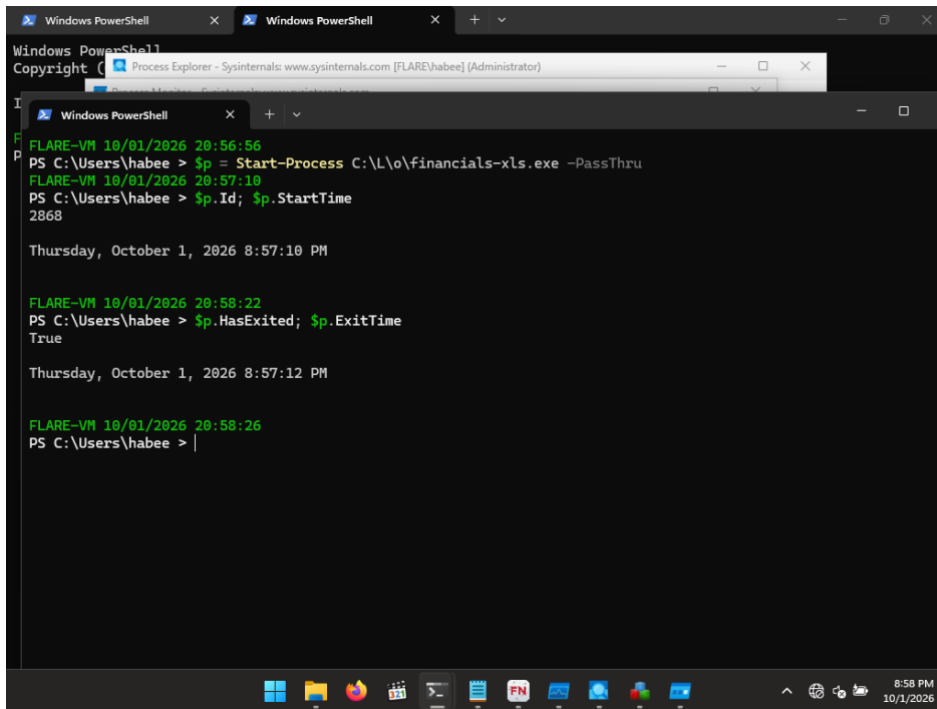


Figure 31. Run 3 process details: \$p.Id = 2868 and StartTime 8:57:10 PM; at prompt 20:58:22, \$p.HasExited = True and ExitTime 8:57:12 PM. [Source: screenshot_32.png]

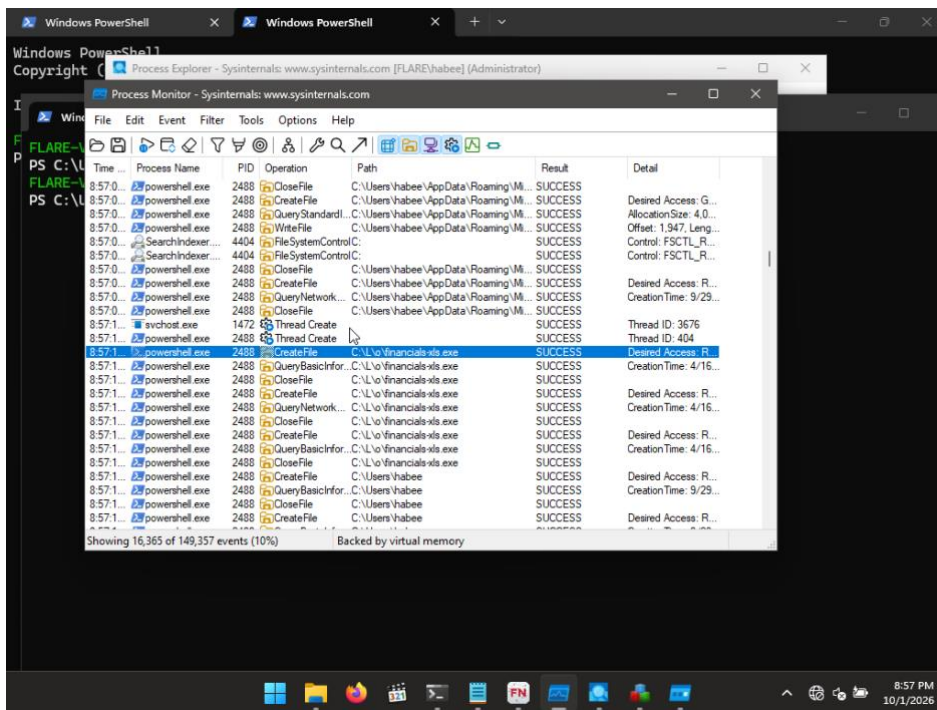


Figure 32. Process Monitor during run 3 (8:57 PM): 16,365 of 149,357 events shown (filtered). Highlighted row: powershell.exe (PID 2488) CreateFile on C:\L\o\financials.xls.exe, followed by QueryBasicInformationFile, CloseFile and QueryNetworkOpenInformationFile on the same path. [Source: screenshot_34.png]

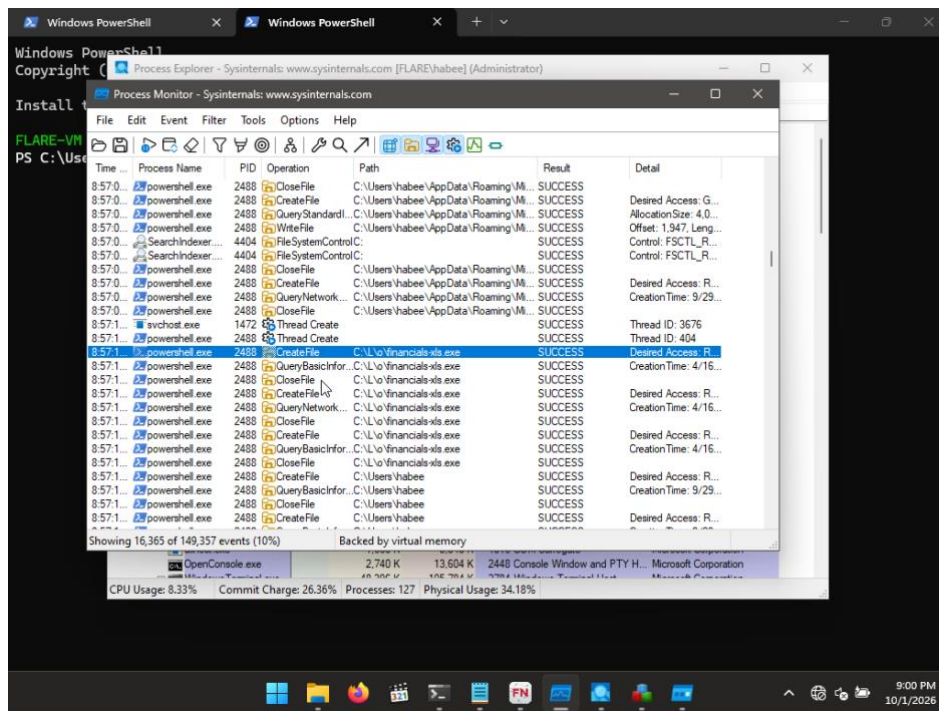


Figure 33. Process Monitor with Process Explorer behind it (9:00 PM), same filtered view (16,365 of 149,357 events). The Procmon capture was not saved to C:\L\e. [Source: screenshot_35.png]

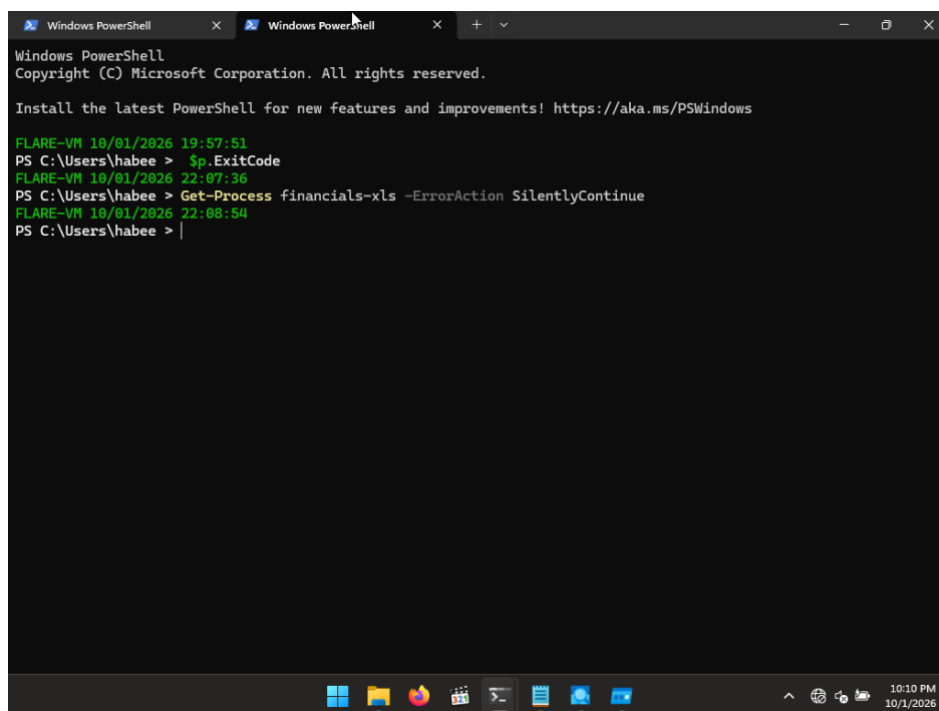


Figure 37. Exit-status checks in a second PowerShell tab (prompts 22:07:36 and 22:08:54): \$p.ExitCode printed nothing because \$p was not defined in this tab; Get-Process financials-xls returned no running process. [Source: screenshot_42b.png]

Process Monitor was running with a filter (16,365 of 149,357 events displayed). The visible rows show the launcher touching the sample file; no rows attributed to financials-xls.exe itself appear in the captured portion. The Procmon log was not saved to C:\L\e, so the process tree, child processes and the sample's own file and registry operations cannot be reviewed.

4.3 File system activity

No file-system activity by the sample was established. Regshot's directory scan was off, so its "files added/modified: 0" result is not a finding (Figure 34), and the Procmon capture was not preserved. The

only file-related observations are that the original sample's hash and size did not change (Section 7) and that no unexpected files are visible in C:\L\o beyond the files the analysis itself created (Figure 41).

4.4 Registry, startup and scheduled-task changes

Observed. Autoruns compared against its 8:27 PM baseline at 9:09 PM shows two new entries on the Everything tab, both under HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run, and no other differences are visible in the capture. A PowerShell comparison of the saved Run-key baseline (b_u.txt, 17:09) with a new export (a_u.txt, 22:04) gives the same two values:

Value name	Type	Data	Notes
con	REG_SZ	C:\L\o\financials-xls.exe	Points at the sample's own path; Autoruns shows the Excel icon, "Not Verified"
Windows update loader	REG_SZ	C:\Windows\xpupdate.exe	Different file name; generic document icon in Autoruns; existence of the target not checked

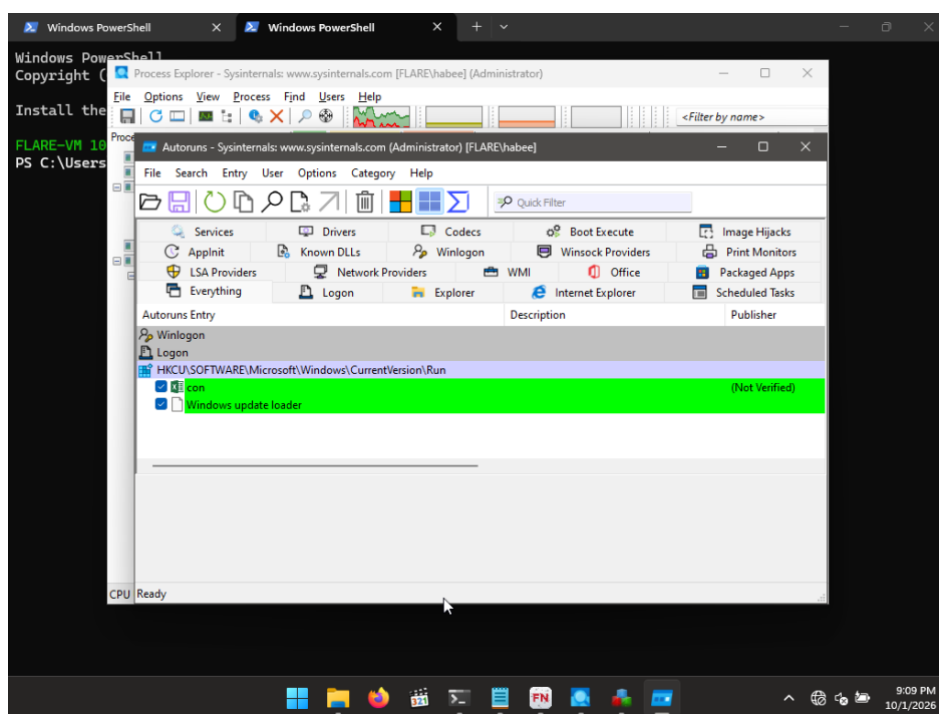


Figure 35. Autoruns comparison against autoruns_before.arn (9:09 PM): the only new entries (green) are under HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run: "con" (Excel icon, Not Verified) and "Windows update loader". [Source: screenshot_37.png]

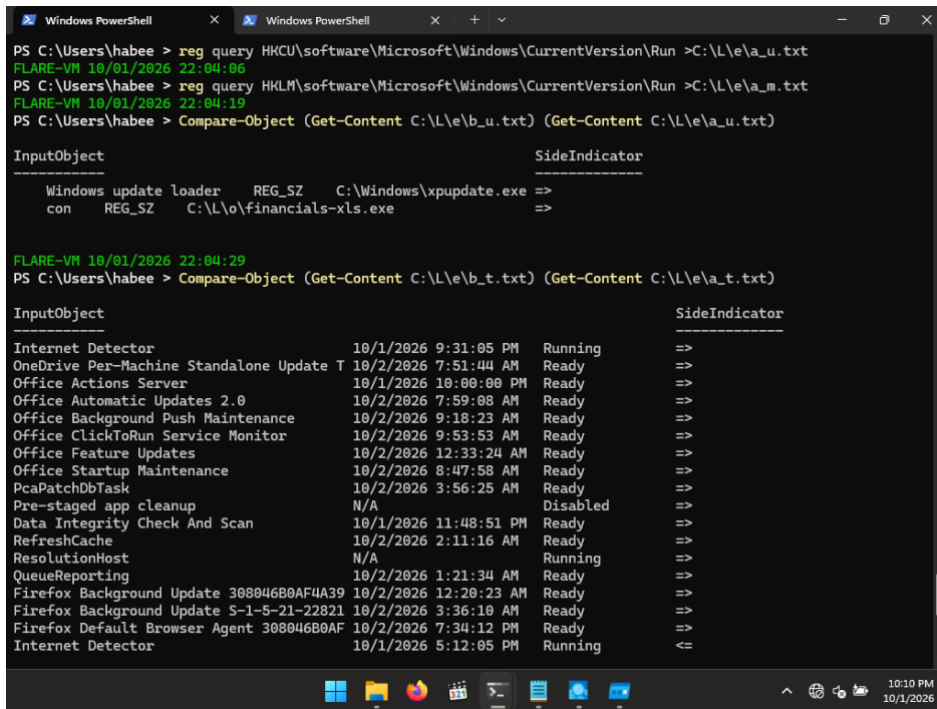


Figure 36. Post-run comparisons (prompts 22:04:06 to 22:04:29): a_u.txt and a_m.txt saved; Compare-Object b_u vs a_u shows two additions, "Windows update loader REG_SZ C:\Windows\xpupdate.exe" and "con REG_SZ C:\L\o\financials.xls.exe". The scheduled-task comparison (partly visible) lists changed next-run times and states for existing tasks. [Source: screenshot_42a.png]

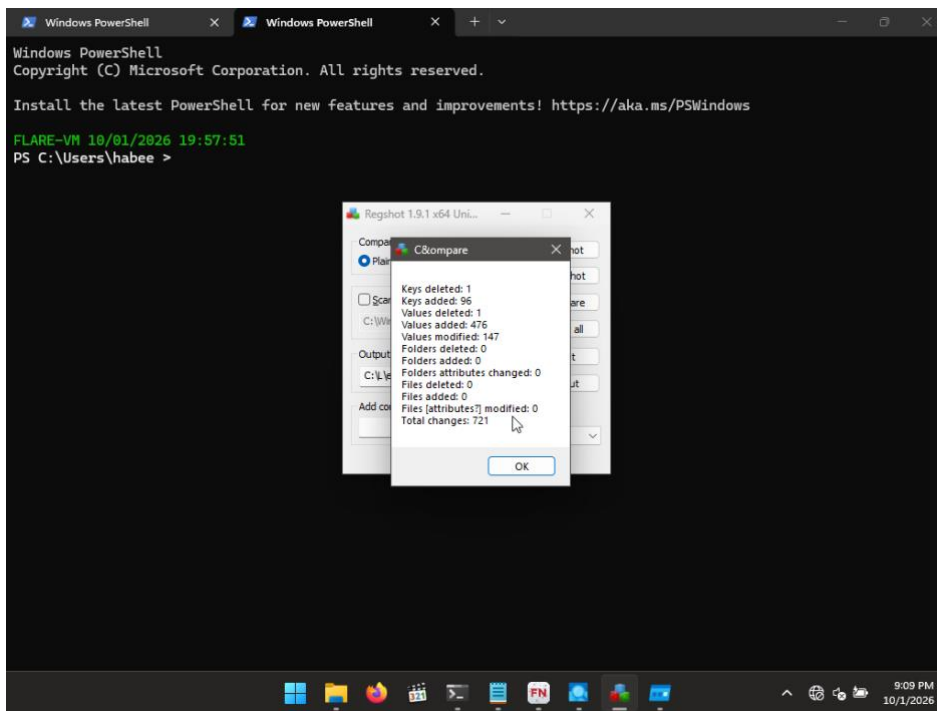


Figure 34. Regshot comparison result (9:09 PM): keys deleted 1, keys added 96, values deleted 1, values added 476, values modified 147, folders and files 0 (directory scan disabled); total changes 721. [Source: screenshot_36.png]

- **Regshot:** 721 total changes (96 keys added, 476 values added, 147 values modified, 1 key and 1 value deleted). These include a lot of background Windows activity during a long window, so they are not attributed to the sample without further evidence.
- **Scheduled tasks:** the comparison output that is visible lists changed next-run times and states for existing Windows, Office and Firefox tasks; no new task was identified in the visible portion. A name-only comparison was not run, so this is inconclusive rather than negative.

- **Services:** the Autoruns comparison shows no new service entries; services were not examined separately.

Interpretation. Both Run values were absent at the baselines (17:09 and 20:27) and present at 21:09 and 22:04, after the three runs. The "con" value points to the sample's own path, which strongly links it to the sample. The "Windows update loader" value links to the sample by timing only. The process that wrote the values was not captured, so authorship is inferred, not proven.

4.5 Absent or inconclusive evidence

Item	Status	Reason
Full process tree and child processes	Not established	Procmon capture not saved; no Process Explorer tree screenshot
File creation, modification or deletion by the sample	Not established	Procmon capture not saved; Regshot directory scan off
Which process wrote the Run values	Not proven	No registry-write events preserved
Existence and content of C:\Windows\xpupdate.exe	Not checked	No file-metadata or hash check was run
New scheduled tasks	Inconclusive	Only part of the comparison output is visible; no name-only comparison
Exit code of run 3	Not obtained	\$p was undefined in the tab used
Server response content	Not analysed	Response came from FakeNet-NG; not reviewed
Anti-analysis or environment checks	Not verified	See Section 5.5
Behaviour on a real network	Unknown	Lab had no Internet access by design

5 Network and behaviour correlation (Section D)

5.1 Capture setup

The VM had no Internet access. Network activity was recorded by FakeNet-NG, whose Diverter logs each connection together with the originating process name and PID and whose listeners answer with simulated content, and by a Wireshark capture saved as packets_20261001_195556.pcap (1,383 KB, last written 9:10 PM). Host-only networking meant nothing could leave the lab.

5.2 Observed connections

Time (VM)	Process (PID)	Protocol and destination	Request seen by HTTPListener80	Outcome
20:49:00	financials-xls.exe (1976)	TCP 69.50.175.181:80	GET /download.php?&advid=00000717&u=2884&p=47377624 HTTP/1.0; Host: download.bravesentry.com	Accepted and logged by FakeNet-NG
20:53:01	financials-xls.exe (1600)	TCP 69.50.175.181:80	GET /download.php?&advid=00000717&u=4326&p=47246552 HTTP/1.0; Host: download.bravesentry.com	Accepted and logged by FakeNet-NG
20:57:11	financials-xls.exe (2868)	TCP 69.50.175.181:80	GET /download.php?&advid=00000717&u=5768&p=47508696 HTTP/1.0; Host: download.bravesentry.com	Accepted and logged by FakeNet-NG

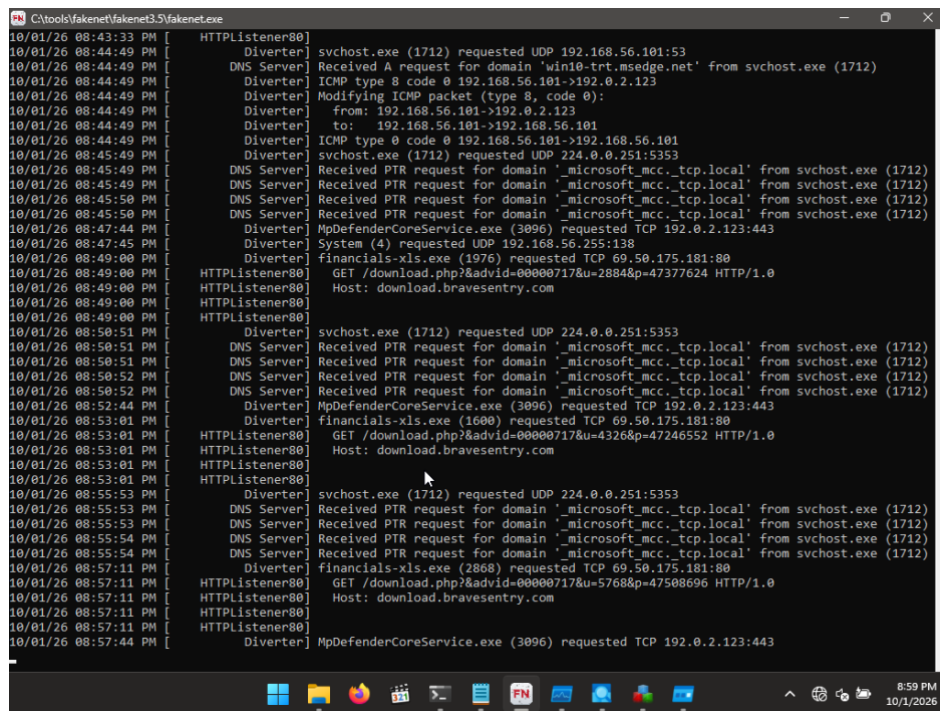


Figure 38. FakeNet-NG console (8:59 PM): three requests from financials-xls.exe at 20:49:00 (PID 1976), 20:53:01 (PID 1600) and 20:57:11 (PID 2868), each "requested TCP 69.50.175.181:80" followed by "GET /download.php?&advid=00000717&u=...&p=... HTTP/1.0" and "Host: download.bravesentry.com". Other lines come from svchost.exe (1712), MpDefenderCoreService.exe (3096) and System (4). [Source: screenshot_33.png]

5.3 Correlation with the process evidence

- The 20:57:11 request is attributed to PID **2868**, which matches the PID of run 3 recorded by PowerShell (\$p.Id = 2868, started 20:57:10, exited 20:57:12). The request fell within the process's two-second life, so correlation is **strong** for run 3.

- The 20:49:00 and 20:53:01 requests carry PIDs 1976 and 1600; these runs have no launch screenshots, so their attribution rests on the FakeNet-NG Diverter log alone.
- Across the three requests the advid value (00000717) is constant and u and p vary. The requests use HTTP/1.0 and show only a Host header in the FakeNet-NG listener log. The purpose of the parameters is unknown.
- Other entries in the same log window belong to the operating system (svchost.exe PID 1712 name-resolution traffic, MpDefenderCoreService.exe PID 3096 to 192.0.2.123:443, and System PID 4 NetBIOS broadcasts) and are excluded from the sample's activity.

5.4 DNS

In the portion of the FakeNet-NG log captured (08:43:33 PM to 08:57:44 PM), no DNS query for download.bravesentry.com or any other name is attributed to financials-xls.exe; all DNS lines in that window are attributed to svchost.exe. The connection was made directly to 69.50.175.181 with the host name appearing only in the HTTP Host header. This is consistent with a hard-coded address, but it rests on one log excerpt and the pcap was not searched for DNS or other sample traffic, so confidence is limited.

5.5 Packet capture, anti-analysis and environment checks

The capture file is preserved in C:\L\e. The one Wireshark view recorded (tcp.stream eq 439) is a FakeNet-NG default web page at 20:37:45, about 11 minutes before the first run; it is not sample traffic and is not used as evidence of sample behaviour. Sample-specific filters (for example on the destination IP or the download.php URI) were not applied, which is a limitation of this report.

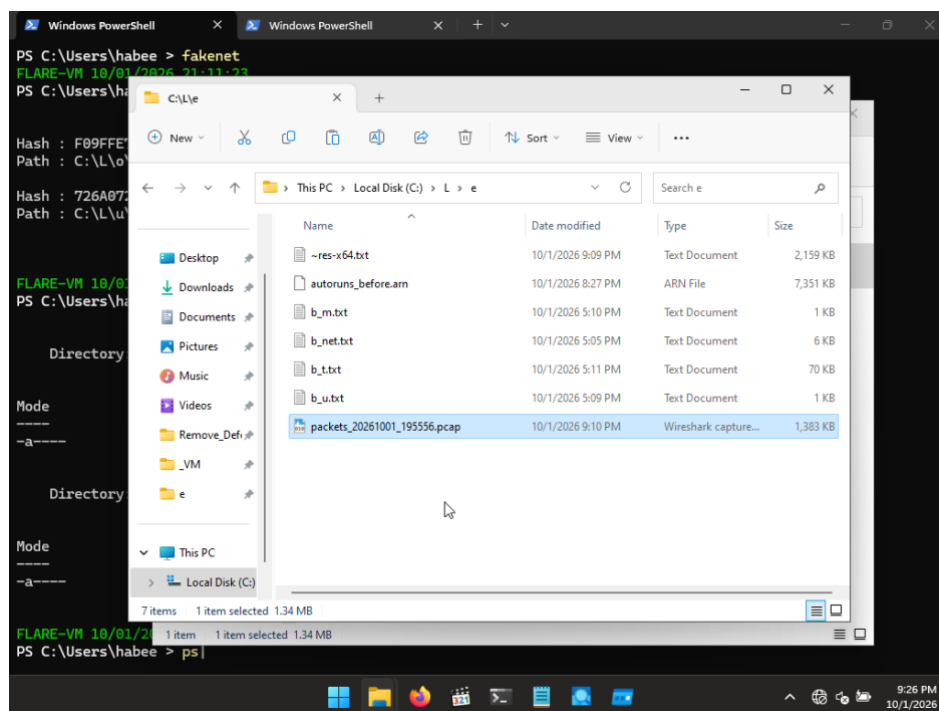


Figure 39. Evidence folder C:\L\e at 9:26 PM: packets_20261001_195556.pcap (1,383 KB, last written 9:10 PM), autoruns_before.arn (8:27 PM), ~res-x64.txt (Regshot output, 9:09 PM) and the baseline files. [Source: screenshot_39.png]

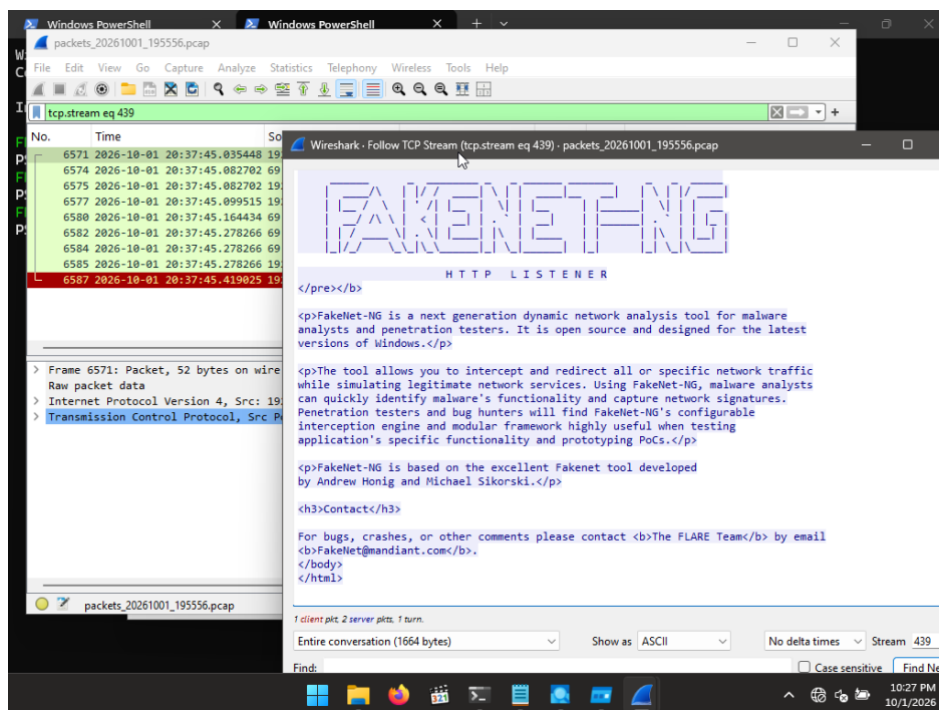


Figure 40. Wireshark Follow TCP Stream (tcp.stream eq 439) in the capture, timestamp 20:37:45, before the first run: the FakeNet-NG default HTTP page. This stream is not sample traffic; it shows the capture was reviewed. [Source: screenshot_43.png]

Anti-analysis behaviour was not verified. Without the Process Monitor capture there is no record of whether the sample queried virtualisation artefacts, debuggers, installed tools or window names. The unpacked import table includes FindWindowA and FindWindowExA, which can be used to look for windows, but that is only a static possibility with no runtime evidence, and the functions are also common in ordinary GUI programs. The sample ran for two seconds, showed its alert and made its request on every run in the analysis VM, so no evidence suggests it refused to run, but absence of evidence is not evidence of absence.

5.6 MITRE ATT&CK mapping

Technique	ID	Supporting evidence	Confidence	Limitation
Obfuscated Files or Information: Software Packing	T1027.002	DIE, pestudio, PE Detective, upx -t [OK], successful unpack (Section 3)	High	Static evidence; UPX is also used by legitimate software, so the mapping describes the technique, not intent
Application Layer Protocol: Web Protocols	T1071.001	Three HTTP GETs to 69.50.175.181:80 with Host download.bravesentry.com (Figure 38)	Medium	Server was simulated; purpose of the exchange (check-in, download, tracking) is unknown
Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder	T1547.001	New HKCU Run values "con" and "Windows update loader" (Figures 35 and 36)	Medium-High (con); Medium (Windows update loader)	Writing process not captured; three runs in the window; xpupdate.exe not verified
Masquerading	T1036	Spreadsheet-style name with Excel icon; Run value named "Windows update loader"	Low-Medium	Based on naming and icon; delivery method unknown

No technique is mapped for the fake infection alert (no fitting technique was identified), for anti-analysis (no evidence) or for execution (the sample was run by the analyst, not by a user or delivery chain).

6 Evidence-based findings (Section E)

6.1 Summary

Observed	Inferred
1. The file shows UPX0/UPX1 sections, UPX1 entropy 7.844, a UPX signature and a passing upx -t test. 2. In run 3 the process (PID 2868) lived about two seconds, showed a fake spyware alert and exited by itself. 3. Each of three runs made one HTTP GET to 69.50.175.181:80 (Host download.bravesentry.com). 4. Two HKCU Run values appeared during the window. 5. The original file's hash was unchanged afterwards.	1. The binary unpacks itself at run time (standard UPX behaviour). 2. The alert is scareware intended to push the user toward "antispware tools". 3. The GET is a check-in or download request to a remote server; its purpose is unknown. 4. Both Run values were written by the sample; authorship is not proven. 5. The unpacked imports (registry, shell notification, sockets) match the observed behaviour but are not proof of how it was achieved.

6.2 Findings matrix

#	Finding	Evidence	ATT&CK	Confidence	Limitation
1	Sample is UPX-packed	DIE UPX(3.92); pestudio UPX0/UPX1, UPX1 entropy 7.844, entry point in UPX1; PE Detective; upx -t [OK]; unpack to 57,344 bytes (Figure 12 to Figure 21)	T1027.002	High	Static; exact UPX version differs by tool
2	Original file unchanged by analysis	Same SHA-256 before, after unpacking and after execution (Figure 8, Figure 21, Figure 42)	None	High	Hash covers file content only
3	Short-lived GUI process showing a fake spyware alert	Figure 30, Figure 31	None	High (observed, run 3)	Runs 1 and 2 not screenshotted; intent (scareware) inferred
4	HTTP GET to hard-coded IP, three times	Figure 38; PID 2868 matches Figure 31	T1071.001	Medium (High that requests occurred)	Simulated server; purpose unknown; runs 1 and 2 attributed by FakeNet-NG only
5	No DNS lookup by the sample observed	Figure 38 (visible window only)	None	Low-Medium	One log excerpt; pcap not searched
6	Run value "con" points to the sample	Figure 35, Figure 36	T1547.001	Medium-High	Writer not captured; three runs in window
7	Run value "Windows update loader" points to C:\Windows\xpupdate.exe	Figure 35, Figure 36	T1547.001, T1036	Medium	Link to sample rests on timing; target file not checked
8	Disguised as a spreadsheet and a Windows update	File name and Excel icon (Figure 7, Figure 35); Run value name	T1036	Low-Medium	Delivery method unknown
9	Imports suggest registry, shell-notification and socket capability	Figure 16, Figure 25	None	Low	Static; scrolled view of 85 imports; not proof of use
10	Anti-analysis checks	Not verified	None	n/a	No Procmon capture; FindWindow imports are a static possibility only
11	File and process activity of the sample	Not established	None	n/a	Procmon capture not preserved; Regshot directory scan off

6.3 Indicators of compromise

Type	Value	Evidence source	Timestamp (VM clock)
File name / size	financials-xls.exe, 43,520 bytes	Figure 7	17:15
SHA-256 (packed)	F09FFE74770A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E5B5A	Figure 8, Figure 21, Figure 42	17:16:37; 19:07:57; 21:23:38
SHA-256 (unpacked)	726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9	Figure 21, Figure 25	19:07:57
Imphash (MD5, packed)	4A5EBEC485BEB64F91EDF76F986F8113	Figure 17 (pestudio)	18:39
IP and port	69.50.175.181:80/TCP	Figure 38 (FakeNet-NG Diverter)	20:49:00; 20:53:01; 20:57:11
HTTP Host	download.bravesentry.com	Figure 38 (HTTPListener80)	20:49:00; 20:53:01; 20:57:11
URI pattern	/download.php?&advid=00000717&u=<varies>&p=<varies> (HTTP/1.0 GET)	Figure 38	20:49:00; 20:53:01; 20:57:11
Registry	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\con = C:\L\o\financials-xls.exe (path reflects the analysis location)	Figure 35, Figure 36	21:09; 22:04:29
Registry	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Windows update loader = C:\Windows\xpupdate.exe	Figure 35, Figure 36	21:09; 22:04:29
On-screen text	"Your computer is infected! Windows has detected spyware infection!"	Figure 30	20:57

Weaker details, not listed as indicators: the PE compile stamp (7 May 2007) and Russian resource language come from a packed file and are easy to alter. The process-ID values and the u and p URI values vary per run and are not reliable indicators.

6.4 Conclusion

financials-xls.exe is a UPX-packed 32-bit Windows GUI executable. Static analysis (Detect It Easy, pestudio, PE Detective and upx -t) and a successful unpack to 57,344 bytes give **high confidence** in the packing result. In three executions inside a Host-only VM with simulated services, the sample ran for about two seconds, displayed a fake spyware-infection alert and sent an HTTP GET to the hard-coded address 69.50.175.181:80 requesting /download.php with a constant advid and varying u and p parameters. Two HKCU Run values appeared during the monitoring window: one pointing at the sample's own path and one pointing at C:\Windows\xpupdate.exe.

The most likely capability is a **scareware or rogue-security-style program that contacts a remote server (possibly to download or register) and sets up Run-key persistence**. Confidence is **medium** overall: high for packing, the alert and the network requests, medium to medium-high for persistence, and **low** for anything a downloaded payload might do, because the server was simulated.

6.5 Limitations

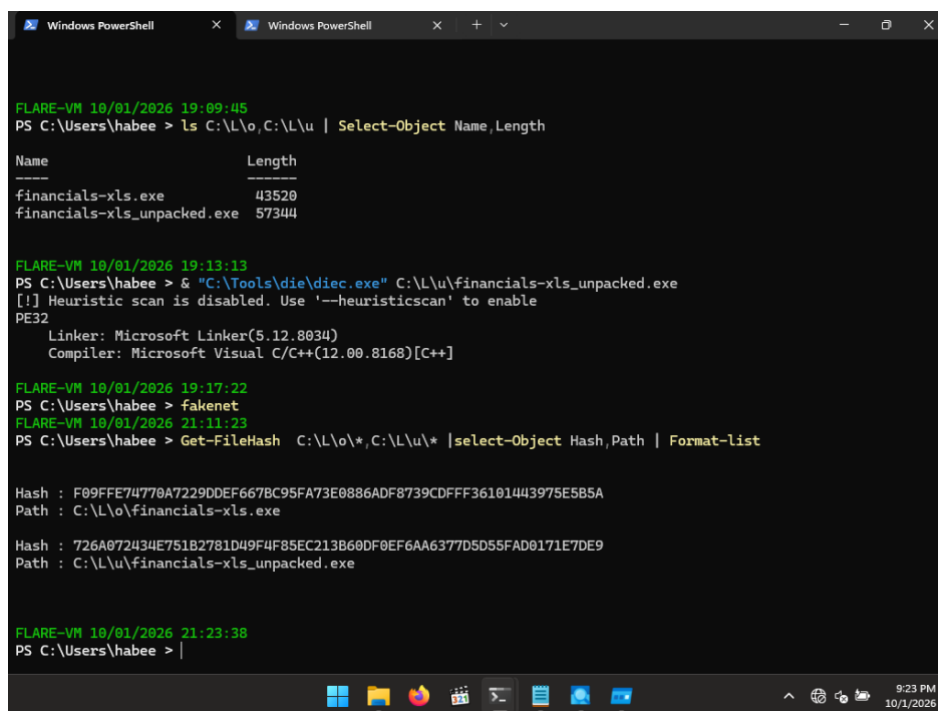
- The Process Monitor capture was not preserved, so the process tree, the sample's own file and registry operations, the writer of the Run values and any anti-analysis checks are unverified.
- The sample ran three times in one window, so Regshot and Autoruns differences cannot be tied to a single run, and only run 3 has launch and exit screenshots.

- The network server was simulated by FakeNet-NG, so real server behaviour, the response content and any second-stage payload are unknown. The pcap was not filtered for sample-specific traffic and the DNS observation rests on one log excerpt.
- Regshot file scanning was disabled, the baseline process list (b_ps.txt) was not saved, and the existence and content of C:\Windows\xpupdate.exe were not checked.
- **Snapshot timing.** VirtualBox time-stamps Lab02_PreExecution_ToolsReady at 9:04 PM (host clock). No capture compares the host and VM clocks, but the PostExecution snapshot (host 10:06 PM) falls between VM events at 22:04 and 22:07, which suggests the two clocks agree to within a few minutes. On that basis 9:04 PM is later than the three runs (20:49 to 20:57), so the snapshot's name and description ("sample not yet executed") are not supported by its timestamp. It is therefore not relied on as a pre-execution restore point; the clean restore point is Lab02_PreExecution_Baseline (3:08 PM).
- The lab asked for a snapshot before the archive was handled. The baseline snapshot precedes folder creation and extraction, but the time the archive reached the VM was not recorded.

7 Post-exercise integrity, preservation and restoration

7.1 Integrity after execution

Re-hashing both files after the runs at 21:23:38 returned the same values as before: the original is still F09FFE74...5E5B5A and the unpacked copy still 726A0724...7DE9, with the same sizes (43,520 and 57,344 bytes). The sample did not modify or delete its own file.



```
FLARE-VM 10/01/2026 19:09:45
PS C:\Users\habee > ls C:\L\o,C:\L\u | Select-Object Name,Length

Name                               Length
----                               -
financials-xls.exe                 43520
financials-xls_unpacked.exe        57344

FLARE-VM 10/01/2026 19:13:13
PS C:\Users\habee > & "C:\Tools\die\diec.exe" C:\L\u\financials-xls_unpacked.exe
[!] Heuristic scan is disabled. Use '--heuristicscan' to enable
PE32
  Linker: Microsoft Linker(5.12.8034)
  Compiler: Microsoft Visual C/C++(12.00.8168)[C++]

FLARE-VM 10/01/2026 19:17:22
PS C:\Users\habee > fakenet
FLARE-VM 10/01/2026 21:11:23
PS C:\Users\habee > Get-FileHash C:\L\o\*,C:\L\u\* |select-Object Hash,Path | Format-List

Hash : F09FFE74778A7229DDEF667BC95FA73E0886ADF8739CDFFF36101443975E585A
Path : C:\L\o\financials-xls.exe
Hash : 726A072434E751B2781D49F4F85EC213B60DF0EF6AA6377D5D55FAD0171E7DE9
Path : C:\L\u\financials-xls_unpacked.exe

FLARE-VM 10/01/2026 21:23:38
PS C:\Users\habee > |
```

Figure 42. Integrity re-check after the runs (command at prompt 21:11:23, completed 21:23:38): original still F09FFE74...5E5B5A and unpacked 726A0724...7DE9; sizes 43,520 and 57,344 bytes. [Source: screenshot_38.png]

7.2 Evidence preservation

Post-run evidence was saved to C:\L\e: a_net.txt (netstat), a_ps.txt (process list) and a_t.txt (scheduled tasks) at 9:29 to 9:30 PM, followed at 22:04 by a_u.txt and a_m.txt (Run keys) for the comparisons in Section 4.4. These post-run files were not compared against baselines except where stated in Section 4.4; the after-lists for processes and network connections were captured but not analysed.

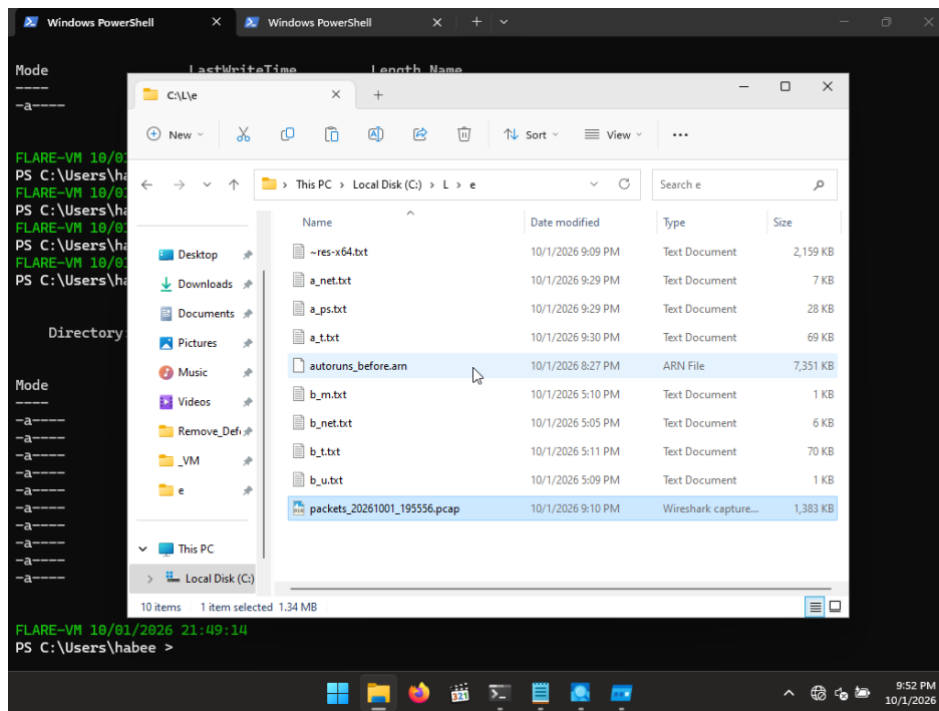


Figure 41. Evidence folder at 9:52 PM: a_net.txt (9:29 PM), a_ps.txt (9:29 PM) and a_t.txt (9:30 PM) added to the baseline files, the pcap and autoruns_before.arn. No Procmon log file and no b_ps.txt are present. [Source: screenshot_41.png]

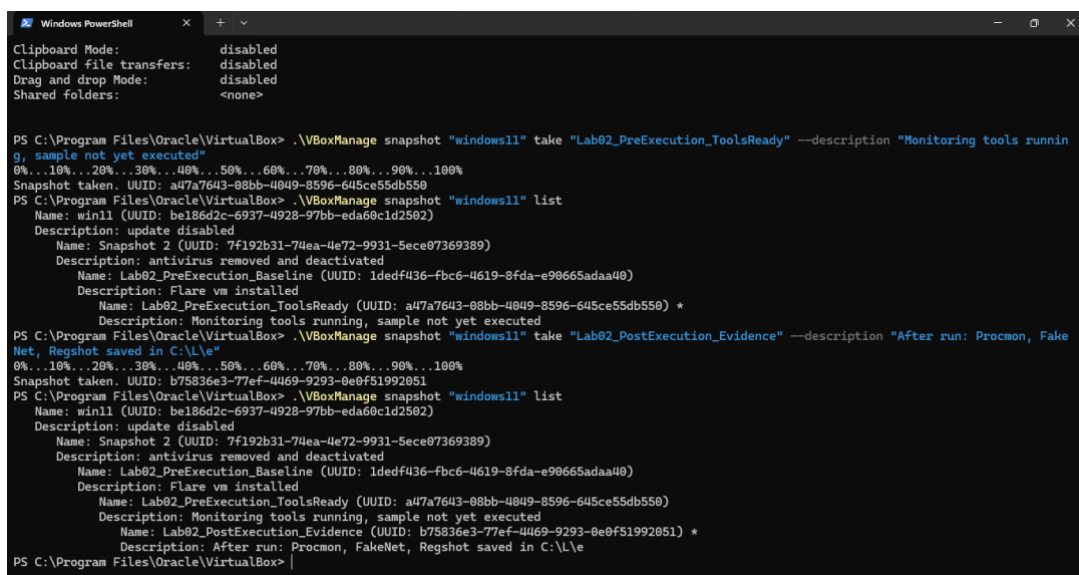


Figure 43. Snapshot Lab02_PostExecution_Evidence taken after the runs ("After run: Procmon, FakeNet, Regshot saved in C:\L\e") and the updated snapshot list. [Source: screenshot_40.png]

7.3 Restoration

At the end of the exercise the VM was powered off and Lab02_PreExecution_Baseline was selected and restored in VirtualBox (about 23:00 host time, from the relative "hours ago" labels in the snapshot panel). The capture shows the restore in progress; later snapshots remain in the tree.

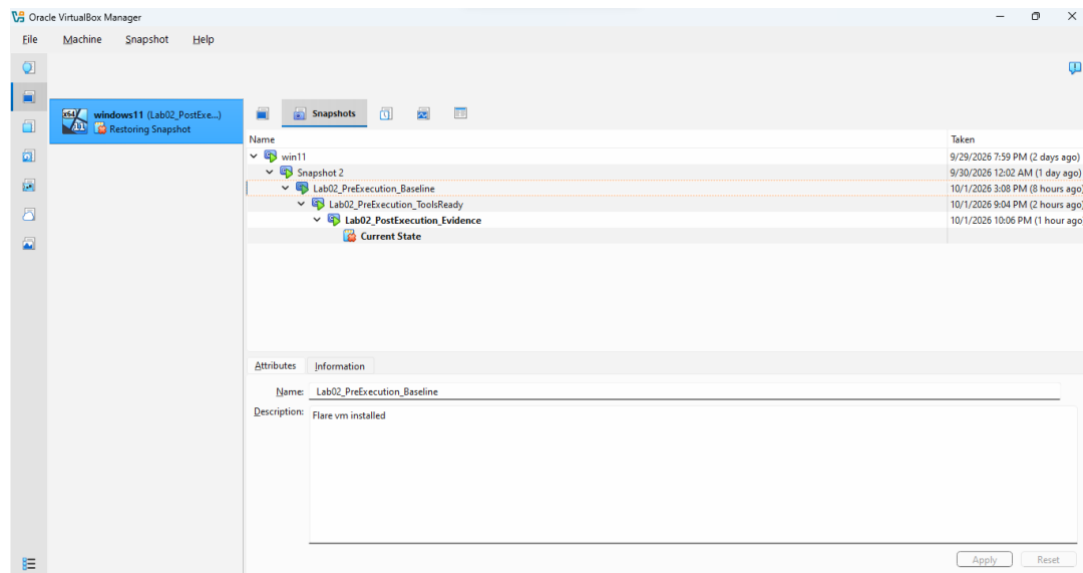


Figure 44. Restore of Lab02_PreExecution_Baseline: VirtualBox Manager shows the VM in the "Restoring Snapshot" state with Lab02_PreExecution_Baseline selected; later snapshots ToolsReady (9:04 PM) and PostExecution_Evidence (10:06 PM) remain in the tree. [Source: Restored_baseline.png]

Appendix A Evidence log

Chronological record linking each artifact to the sample copy, timestamp and tool. Times are VM clock unless marked host. Ranges cover a group of related commands.

Time	Action and tool	Sample copy	Artifact	Why
host (not shown)	Isolation check; first attempt returned a FindMachine error, then re-run with the correct VM name. <code>VBBoxManage showvminfo "windows11"</code>	n/a	Figure 1	Prove clipboard, drag-and-drop and shared folders are off and that only Host-only networking is attached
host (not shown)	VM configuration summary. VirtualBox Manager	n/a	Figure 2	Record VM name, OS, memory and network adapter
host (not shown)	Settings check. VirtualBox Settings	n/a	Figure 3	Visual proof of clipboard and drag-and-drop settings
15:08 (host)	Baseline snapshot Lab02_PreExecution_Baseline. VirtualBox	n/a	Figure 4	Clean restore point
16:52	Created C:\L\o and C:\L\e (C:\L\o was created later, before the unpack). <code>md</code>	n/a	Figure 5	Separate original, evidence and unpacked files
16:55:54	Recorded environment details. <code>hostname; cmd /c ver; date</code>	n/a	Figure 5	OS version and time reference
17:05 to 17:11	Baselines. <code>netstat -ano, reg query (HKCU and HKLM Run), schtasks; process list</code> shown on screen only	n/a	Figure 6; b_net, b_u, b_m, b_t	Before-state for comparison; b_ps.txt not saved
17:15	Sample present in C:\L\o. File Explorer	original	Figure 7	Confirm extraction without execution
17:16:37	SHA-256 of the original (first attempt mistyped). <code>Get-FileHash</code>	original	Figure 8	Chain of custody
17:35 to 17:42	UPX version; PATH check. <code>upx -v, where.exe</code>	n/a	Figure 9	Confirm approved tools; diec not on PATH
17:42 to 17:59	Located DIE files. <code>gci</code>	n/a	Figure 10	Find die.exe and diec.exe in C:\Tools\die
18:07 to 18:11	DIE command line (one typo, one Ctrl+C). <code>diec.exe</code>	original	Figure 11	Packer identification by signature
18:14	DIE GUI scan. Detect It Easy v3.10	original	Figure 12	Signature plus heuristic packing detection
18:24 to 18:25	DIE entropy and sections. Detect It Easy	original	Figure 13, Figure 14	Entropy and section evidence
18:39 to 18:43	pestudio indicators, imports and sections	original	Figure 17, Figure 16, Figure 15	Independent structure, entropy and import evidence
18:45	PE Detective deep scan	original	Figure 18	Independent signature check
18:46 to 18:47	UPX test and listing. <code>upx -t, upx -l</code>	original	Figure 19	Packer confirms packing; no execution
about 19:06	Approved unpack. <code>upx -d -o</code>	original to unpacked	Figure 20	Create financials-xls_unpacked.exe; original untouched
19:07:57	Hash both files (an earlier command was interrupted at 19:06:41). <code>Get-FileHash</code>	both	Figure 21	Traceable hashes; original unchanged
19:09 to 19:17	Sizes and DIE scan of unpacked file. <code>ls, diec.exe</code>	unpacked	Figure 22	Size change; packer no longer reported
19:24 to 19:25	DIE entropy and sections of unpacked file	unpacked	Figure 23, Figure 24	Entropy and section comparison

Time	Action and tool	Sample copy	Artifact	Why
19:29	pestudio imports of unpacked file	unpacked	Figure 25	Import visibility after unpacking
19:55:56 (from file name)	Packet capture started. Wireshark	n/a	Figure 39; packets_20261001_195556.pcap	Record network traffic during the session
before 20:49	Isolation re-check. VBoxManage showvminfo	n/a	Figure 26	Confirm controls still intact before execution
20:25	Regshot configured (file scan off). Regshot 1.9.1	n/a	Figure 28	Prepare registry comparison
20:27	Autoruns baseline saved. Autoruns	n/a	Figure 29; autoruns_before.arn	Autostart baseline
20:49:00	Run 1 network event (PID 1976). FakeNet-NG	original	Figure 38	Observed connection to 69.50.175.181:80
20:53:01	Run 2 network event (PID 1600). FakeNet-NG	original	Figure 38	Observed connection to 69.50.175.181:80
20:56:56 to 20:57:10	Run 3 launched. Start-Process ... - PassThru; fake alert displayed	original	Figure 30	Record command line and visible behaviour
20:57:10 to 20:57:12	Run 3 PID, start and exit. \$p.Id; \$p.StartTime; \$p.HasExited; \$p.ExitTime	original	Figure 31	Process ID and lifetime
20:57:11	Run 3 network event (PID 2868). FakeNet-NG	original	Figure 38	Correlate connection with PID 2868
20:57 and 21:00	Process Monitor and Process Explorer views	original	Figure 32, Figure 33	Launcher activity; capture not saved
21:04 (host)	Snapshot Lab02_PreExecution_ToolsReady. VBoxManage snapshot take	n/a	Figure 27	Tools-ready state (timing caveat in 6.5)
21:09	Regshot comparison	n/a	Figure 34; ~res-x64.txt	Registry change summary
21:09	Autoruns comparison	n/a	Figure 35	New autostart entries
21:11 to 21:23:38	Re-hash after execution. Get-FileHash	both	Figure 42	Confirm files unchanged
21:26 and 21:52	Evidence folder listings. File Explorer	n/a	Figure 39, Figure 41	Show preserved evidence files
21:29 to 21:30	Post-run netstat, process list, tasks saved to a_net, a_ps, a_t	n/a	Figure 41	After-state files
22:04:06 to 22:04:29	Run-key exports and comparisons. reg query, Compare-Object	n/a	Figure 36	Confirm persistence values and task changes
22:06 (host)	Snapshot Lab02_PostExecution_Evidence	n/a	Figure 43	Preserve post-run state
22:07:36 and 22:08:54	Exit-code and process checks (wrong tab for \$p)	original	Figure 37	Confirm no running instance
22:27	Wireshark Follow TCP Stream, tcp.stream eq 439 (time 20:37:45)	n/a	Figure 40	Not sample traffic; recorded for completeness
about 23:00 (host)	Restore Lab02_PreExecution_Baseline. VirtualBox	n/a	Figure 44	Return VM to clean state

Appendix B Figure index

Every figure is embedded in the section it supports. The source file names below are the exported screenshots in the evidence package.

Figure	Source file	Section
1	baseline_configuration_of_the_windows_11_from_powershell.png	2 Environment
2	baseline_configuration_of_the_windows_11.png	2 Environment
3	screenshot_2.png	2 Environment
4	Lab02_PreExecution_Baseline.png	2 Environment
5	screenshot_6.png	2 Environment
6	screenshot_7.png	2 Environment
7	Extracted_malware_to_the_original_folder.png	2 Environment
8	Original_malware_file_hashed.png	2 Environment
9	screenshot_9.png	3 Packing
10	screenshot_10.png	3 Packing
11	screenshot_11.png	3 Packing
12	screenshot_12.png	3 Packing
13	screenshot_13.png	3 Packing
14	screenshot_14.png	3 Packing
15	screenshot_15.png	3 Packing
16	screenshot_16.png	3 Packing
17	screenshot_17.png	3 Packing
18	screenshot_18.png	3 Packing
19	screenshot_19.png	3 Packing
20	screenshot_20.png	3 Packing
21	screenshot_21.png	3 Packing
22	screenshot_22.png	3 Packing
23	screenshot_23.png	3 Packing
24	screenshot_23b.png	3 Packing
25	screenshot_24.png	3 Packing
26	Screenshot_29.png	4 Runtime
27	screenshot_30.png	4 Runtime
28	screenshot_27.png	4 Runtime
29	screenshot_28.png	4 Runtime
30	screenshot_31.png	4 Runtime
31	screenshot_32.png	4 Runtime
32	screenshot_34.png	4 Runtime
33	screenshot_35.png	4 Runtime
34	screenshot_36.png	4 Runtime
35	screenshot_37.png	4 Runtime
36	screenshot_42a.png	4 Runtime

Figure	Source file	Section
37	screenshot_42b.png	4 Runtime
38	screenshot_33.png	5 Network
39	screenshot_39.png	5 Network
40	screenshot_43.png	5 Network
41	screenshot_41.png	7 Post-exercise
42	screenshot_38.png	7 Post-exercise
43	screenshot_40.png	7 Post-exercise
44	Restored_baseline.png	7 Post-exercise

Appendix C Deviations, errors and corrections

These are recorded for transparency; none changes the conclusions.

Item	What happened	Effect
VBoxManage first attempt	FindMachine error, then re-run with VM name windows11	None; second run shown in the figures
Network adapter	Host-only is on NIC 2; NIC 1 is disabled	Report states NIC 2
PE-bear	Not installed; PE Carve present but not used	pestudio and DIE used for sections and imports
diec not on PATH	where.exe could not find it; full-path call used	None
Mistyped commands	gGet-FileHash (17:16:37); finacials.xls.exe (18:07:38); Ctrl+C interruptions at 18:07:27 and 19:06:41	Commands re-run correctly
Baseline process list	Displayed on screen; b_ps.txt not saved	No saved process baseline
Procmon capture	Running during execution but no log saved	Process and file activity unverified
Regshot	Directory scan disabled	File results not meaningful
Three runs in one window	Only run 3 has launch and exit screenshots	Changes cannot be tied to a single run
Exit code	\$p.ExitCode typed in a tab where \$p was undefined	Exit confirmed instead by HasExited and ExitTime
Wireshark stream 439	Shows FakeNet-NG default page at 20:37:45, before the first run	Not used as sample evidence
Optional checks	xpupdate.exe existence, name-only task comparison and sample-specific Wireshark filters were not performed	Listed as limitations
ToolsReady snapshot timing	Host time stamp 9:04 PM is later than the recorded runs	Not relied on as pre-execution; see 6.5
Snapshot description	Baseline description reads "Flare vm installed"	Quoted as entered